```
================================================================================
DEFMAN-2026: THIRD NORMAL FORM (3NF) DATABASE MIGRATION & ARCHITECTURE DOCUMENT
================================================================================
Project:      DefMan (Defect Manager) -- 2026 Edition
Database:     MySQL 8.0+ / MariaDB 10.6+ (InnoDB, utf8mb4_unicode_ci)
Backend:      PHP 8.3+ Strict-Typed PDO REST API
Migration:    5 Legacy Denormalized Tables -> 10 Strict 3NF Relational Tables
              + 3 Backward-Compatibility SQL Views (‘product‘, ‘raport‘, ‘analyse‘)
================================================================================

================================================================================
SECTION 1. EXECUTIVE SUMMARY & NORMALIZATION ANALYSIS (1NF, 2NF, 3NF)
================================================================================

1.1 Overview of Legacy Schema Violations
--------------------------------------------------------------------------------
The legacy DefMan schema consisted of 5 tables (‘product‘, ‘users‘,
‘user_sessions‘, ‘raport‘, and ‘analyse‘) inherited from the 2004 schema design.
While functional, it contained several First, Second, and Third Normal Form
(1NF, 2NF, 3NF) anomalies as well as data-type and primary-key scalability
bottlenecks:

  A. Naming & Primary Key Exhaustion Risks:
     - Mixed-case and Polish/Finnish legacy identifiers (‘raport‘, ‘analyse‘,
       ‘Product‘, ‘CustomerID‘, ‘Whenreported‘, ‘Discoveredby‘, ‘Occured‘).
     - ‘raport.ID‘ and ‘analyse.id‘ used ‘SMALLINT UNSIGNED‘, capping total
       lifetime defect reports at 65,535 rows.

  B. First Normal Form (1NF) Violations:
     - Repeating Groups / Multi-Valued Column: ‘analyse.phase‘ stored multiple
       SDLC phases as a comma-delimited string (e.g., ’Implementation,Testing’),
       preventing indexed phase queries and referential integrity.
     - Non-Atomic String Timestamps & Redundant Dates: ‘raport.Whenreported‘
       stored timestamps as ‘VARCHAR(32)‘ (’YYYY-MM-DD HH:mm’), while
       ‘raport.Submitted‘ and ‘analyse.submitted‘ duplicated the date portion
       across two tables.

  C. Second & Third Normal Form (2NF / 3NF) Violations:
     - Unnormalized Organization Attribute in ‘users‘: ‘users.company‘ repeated
       free-text company names across user accounts.
     - Transient Session & Recovery Attributes on ‘users‘: ‘users.logged‘
       duplicated session state already tracked in ‘user_sessions‘, causing
       stale "Online" badges when sessions expired. Similarly,
       ‘reset_token_hash‘ and ‘reset_token_expires_at‘ embedded 1:N recovery
       lifecycle data directly inside the ‘users‘ entity.
     - Denormalized Product Name in ‘raport‘: ‘raport.Product‘ (‘VARCHAR(100)‘)
       duplicated ‘product.name‘ alongside a nullable ‘product_id‘ FK.
     - Transitive Reporter Identity in ‘raport‘: ‘raport.Who‘ and
       ‘raport.Contactemail‘ duplicated ‘users.login‘ and ‘users.email‘ even
       when ‘CustomerID‘ referenced a valid registered user.
     - Unnormalized Fault Taxonomy & Inconsistent Enums in ‘analyse‘:
       ‘analyse.classification‘ stored repeated IEEE 1044-1992 strings;
       priority enums differed between ‘raport.Priority‘ (’m_urgent’,
       ’average’, ’l_urgent’) and ‘analyse.priority‘ (’most urgent’,
       ’average’, ’less urgent’); and ‘analyse.uniqueness‘ conflated defect
       duplication (’New defect’, ’Already submitted’) with severity (’Critical’).

--------------------------------------------------------------------------------
1.2 Target 3NF Relational Model (10 Normalized Tables)
--------------------------------------------------------------------------------
  1. ‘products‘              : Standardized from ‘product‘ (‘id‘, ‘name‘, ‘created_at‘).
  2. ‘companies‘             : Extracted 3NF lookup table (‘id‘, ‘name‘, ‘created_at‘).
  3. ‘users‘                 : Normalized user accounts referencing ‘companies(id)‘
                               via ‘company_id‘; ‘logged‘ and reset columns removed.
  4. ‘user_sessions‘         : Active bearer tokens; derives user online presence.
```

```
  5. ‘password_resets‘       : Decoupled password recovery tokens referencing ‘users(id)‘.
  6. ‘sdlc_phases‘           : 1NF lookup table for SDLC phases (’Requirements’,
                               ’Design’, ’Implementation’, ’Testing’, ’Deployment’,
                               ’Maintenance’).
  7. ‘fault_classifications‘ : 3NF lookup table for IEEE 1044-1992 fault codes.
  8. ‘defect_reports‘        : Standardized from ‘raport‘ with ‘INT UNSIGNED‘ PK,
                               ‘product_id‘ NOT NULL FK (‘ON DELETE RESTRICT’),
                               ‘reporter_user_id‘ FK (‘ON DELETE SET NULL‘),
                               nullable ‘guest_name‘ / ‘guest_email‘ enforced by
                               ‘chk_defect_reports_reporter_identity‘, and ‘DATETIME‘
                               ‘reported_at‘.
  9. ‘defect_analyses‘       : Standardized from ‘analyse‘ with ‘INT UNSIGNED‘
                               ‘report_id‘ PK/FK, ‘classification_id‘ FK,
                               ‘uniqueness‘ ENUM(’new’,’duplicate’),
                               ‘duplicate_of_report_id‘ self-referencing FK, and
                               unified ‘priority‘ ENUM(’most_urgent’,’average’,’less_urgent’).
 10. ‘analysis_phases‘       : 1NF junction table (‘report_id‘, ‘phase_id‘) linking
                               ‘defect_analyses‘ to ‘sdlc_phases‘.

--------------------------------------------------------------------------------
1.3 Legacy-to-3NF Schema Mapping Matrix
--------------------------------------------------------------------------------
Legacy Table.Column              -> Target 3NF Table.Column (Type / Constraint)
--------------------------------------------------------------------------------
product.id                       -> products.id (SMALLINT UNSIGNED PK)
product.name                     -> products.name (VARCHAR(100) UK NOT NULL)
users.company                    -> companies.name + users.company_id (FK)
users.logged                     -> Derived via EXISTS(user_sessions.expires_at > NOW())
users.reset_token_hash           -> password_resets.token_hash (CHAR(64) UK)
users.reset_token_expires_at     -> password_resets.expires_at (DATETIME)
raport.ID                        -> defect_reports.id (INT UNSIGNED PK)
raport.Product + product_id      -> defect_reports.product_id (SMALLINT UNSIGNED NOT NULL FK)
raport.Environment               -> defect_reports.environment (TEXT NULL)
raport.CustomerID                -> defect_reports.reporter_user_id (SMALLINT UNSIGNED NULL FK)
raport.Who                       -> users.login (if user) OR defect_reports.guest_name
raport.Contactemail              -> users.email (if user) OR defect_reports.guest_email
raport.Whenreported + Submitted  -> defect_reports.reported_at (DATETIME NOT NULL)
raport.Title                     -> defect_reports.title (VARCHAR(255) NOT NULL)
raport.Description               -> defect_reports.description (TEXT NOT NULL)
raport.Discoveredby              -> defect_reports.discovered_by (VARCHAR(100) NULL)
raport.Persontype                -> defect_reports.person_type (ENUM)
raport.Occured                   -> defect_reports.occurred_on (DATE NOT NULL)
raport.Location                  -> defect_reports.location (VARCHAR(255) NULL)
raport.Severity                  -> defect_reports.initial_severity (ENUM)
raport.Priority                  -> defect_reports.initial_priority (ENUM: most_urgent/average/l
    ess_urgent)
raport.Addinfo                   -> defect_reports.additional_info (TEXT NULL)
analyse.id                       -> defect_analyses.report_id (INT UNSIGNED PK, FK)
analyse.devdesc                  -> defect_analyses.developer_description (TEXT NULL)
analyse.classification           -> fault_classifications.code + defect_analyses.classification_
    id (FK)
analyse.dateoriginated           -> defect_analyses.date_originated (DATE NULL)
analyse.phase                    -> sdlc_phases.name + analysis_phases (report_id, phase_id)
analyse.actoriginated            -> defect_analyses.activity_originated (TEXT NULL)
analyse.deliverable              -> defect_analyses.deliverable (VARCHAR(255) NULL)
analyse.whattodo                 -> defect_analyses.what_to_do (TEXT NULL)
analyse.submitted                -> Removed (derived from defect_reports.reported_at)
analyse.assigned                 -> defect_analyses.assigned_on (DATE NULL)
analyse.uniqueness               -> defect_analyses.uniqueness (ENUM: new/duplicate)
                                    + defect_analyses.duplicate_of_report_id (FK)
analyse.priority                 -> defect_analyses.priority (ENUM: most_urgent/average/less_urg
    ent)
analyse.severity                 -> defect_analyses.severity (ENUM: minor/major/critical)
analyse.workaround               -> defect_analyses.workaround (TEXT NULL)
analyse.proposedfix              -> defect_analyses.proposed_fix (TEXT NULL)
```

```
analyse.status                  -> defect_analyses.status (ENUM)
analyse.fixdesc                 -> defect_analyses.fix_description (TEXT NULL)
analyse.fixstarted              -> defect_analyses.fix_started_on (DATE NULL)
analyse.fixcomplete             -> defect_analyses.fix_completed_on (DATE NULL)
analyse.fiximpact               -> defect_analyses.fix_impact (TEXT NULL)
analyse.fixeffort               -> defect_analyses.fix_effort (DECIMAL(8,2) UNSIGNED)
analyse.fixcost                 -> defect_analyses.fix_cost (DECIMAL(10,2) UNSIGNED)
analyse.resodesc                -> defect_analyses.resolution_description (TEXT NULL)
analyse.resocomplete            -> defect_analyses.resolution_completed_on (DATE NULL)
analyse.resoclosed              -> defect_analyses.resolution_closed_on (DATE NULL)
analyse.costofdefect            -> defect_analyses.cost_of_defect (DECIMAL(10,2) UNSIGNED)
analyse.actfound                -> defect_analyses.activity_found (TEXT NULL)

--------------------------------------------------------------------------------
1.4 Execution Order for Existing Databases vs. Fresh Deployments
--------------------------------------------------------------------------------
  * Existing Legacy Database Migration:
      1. mysql -u root -p opr0413 < database/02_migrate_data.sql
      2. mysql -u root -p opr0413 < database/03_legacy_views.sql

  * Fresh Database Provisioning:
      1. mysql -u root -p opr0413 < database/schema.sql
      2. php backend/bin/seed.php
================================================================================


================================================================================
SECTION 2.1 Target 3NF DDL Script (database/01_schema_3nf.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\01_schema_3nf.sql
================================================================================

-- ============================================================================
-- File: 01_schema_3nf.sql
-- Description: Target 3NF Relational Schema for DefMan-2026 (MySQL 8.0+ / MariaDB 10.6+)
-- Engine: InnoDB | Charset: utf8mb4 | Collation: utf8mb4_unicode_ci
-- ============================================================================

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE = 'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_E
    NGINE_SUBSTITUTION';

-- ----------------------------------------------------------------------------
-- 1. Table: `products` (standardized from `product`)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `products` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_products_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------------------------
-- 2. Table: `companies` (extracted from `users.company` -- 3NF)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `companies` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_companies_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------------------------
```

```
-- 3. Table: ‘users‘ (normalized: ‘company_id‘ FK; removed ‘logged‘ & reset cols)
-- User Levels:
--   0 = Applying (Pending Admin Approval)
--   1 = Public   (Unauthenticated / Public reporter)
--   2 = Customer (Authenticated defect reporter)
--   3 = Analyzer (Developer / QA triage & IEEE fault cost tracking)
--   4 = Admin    (Full system access, user approval & product CRUD)
-- --------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘users‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘login‘ VARCHAR(64) NOT NULL,
  ‘password‘ VARCHAR(255) NOT NULL COMMENT ’Argon2id password hash (PASSWORD_ARGON2ID)’,
  ‘email‘ VARCHAR(255) NOT NULL,
  ‘userlevel‘ TINYINT UNSIGNED NOT NULL DEFAULT 0 COMMENT ’0=Applying..4=Admin’,
  ‘company_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL,
  ‘phone‘ VARCHAR(50) NULL DEFAULT NULL,
  ‘name‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘activate‘ TINYINT(1) NOT NULL DEFAULT 0 COMMENT ’0=Pending/Inactive, 1=Active’,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ‘updated_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_users_login‘ (‘login‘),
  UNIQUE KEY ‘uk_users_email‘ (‘email‘),
  KEY ‘idx_users_company_id‘ (‘company_id‘),
  KEY ‘idx_users_level_activate‘ (‘userlevel‘, ‘activate‘),
  CONSTRAINT ‘fk_users_company‘
    FOREIGN KEY (‘company_id‘) REFERENCES ‘companies‘ (‘id‘)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------
-- 4. Table: ‘user_sessions‘ (active rows derive user online status)
-- --------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘user_sessions‘ (
  ‘id‘ BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘user_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘token_hash‘ CHAR(64) NOT NULL COMMENT ’SHA-256 hash of 256-bit random_bytes token’,
  ‘ip_address‘ VARCHAR(45) NULL DEFAULT NULL,
  ‘user_agent‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘expires_at‘ DATETIME NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_user_sessions_token_hash‘ (‘token_hash‘),
  KEY ‘idx_user_sessions_user_expires‘ (‘user_id‘, ‘expires_at‘),
  KEY ‘idx_user_sessions_expires_at‘ (‘expires_at‘),
  CONSTRAINT ‘fk_user_sessions_user‘
    FOREIGN KEY (‘user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------
-- 5. Table: ‘password_resets‘ (decoupled from ‘users‘ -- 3NF)
-- --------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘password_resets‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘user_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘token_hash‘ CHAR(64) NOT NULL COMMENT ’SHA-256 hash of recovery token’,
  ‘expires_at‘ DATETIME NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_password_resets_token_hash‘ (‘token_hash‘),
  KEY ‘idx_password_resets_user_expires‘ (‘user_id‘, ‘expires_at‘),
  CONSTRAINT ‘fk_password_resets_user‘
    FOREIGN KEY (‘user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 6. Table: ‘sdlc_phases‘ (lookup table for SDLC origin phases -- 1NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘sdlc_phases‘ (
  ‘id‘ TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(64) NOT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_sdlc_phases_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 7. Table: ‘fault_classifications‘ (IEEE 1044-1992 fault classification -- 3NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘fault_classifications‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘code‘ VARCHAR(100) NOT NULL,
  ‘description‘ VARCHAR(255) NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_fault_classifications_code‘ (‘code‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 8. Table: ‘defect_reports‘ (standardized & normalized from ‘raport‘)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘defect_reports‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘product_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘environment‘ TEXT NULL DEFAULT NULL,
  ‘reporter_user_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL,
  ‘guest_name‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘guest_email‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘reported_at‘ DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ‘title‘ VARCHAR(255) NOT NULL,
  ‘description‘ TEXT NOT NULL,
  ‘discovered_by‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘person_type‘ ENUM(’Customer’, ’Developer’, ’Tester’) NOT NULL DEFAULT ’Customer’,
  ‘occurred_on‘ DATE NOT NULL,
  ‘location‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘initial_severity‘ ENUM(’minor’, ’major’, ’critical’) NOT NULL DEFAULT ’minor’,
  ‘initial_priority‘ ENUM(’most_urgent’, ’average’, ’less_urgent’) NOT NULL DEFAULT ’average’,
  ‘additional_info‘ TEXT NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  KEY ‘idx_defect_reports_product_id‘ (‘product_id‘),
  KEY ‘idx_defect_reports_reporter_user_id‘ (‘reporter_user_id‘),
  KEY ‘idx_defect_reports_reported_at‘ (‘reported_at‘ DESC),
  KEY ‘idx_defect_reports_severity_reported‘ (‘initial_severity‘, ‘reported_at‘ DESC),
  KEY ‘idx_defect_reports_priority_reported‘ (‘initial_priority‘, ‘reported_at‘ DESC),
  CONSTRAINT ‘fk_defect_reports_product‘
    FOREIGN KEY (‘product_id‘) REFERENCES ‘products‘ (‘id‘)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT ‘fk_defect_reports_reporter‘
    FOREIGN KEY (‘reporter_user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT ‘chk_defect_reports_reporter_identity‘
    CHECK (
      ‘reporter_user_id‘ IS NOT NULL
      OR (‘guest_name‘ IS NOT NULL AND ‘guest_email‘ IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 9. Table: ‘defect_analyses‘ (standardized & normalized from ‘analyse‘)
-- ------------------------------------------------------------------------
```

```
CREATE TABLE IF NOT EXISTS ‘defect_analyses‘ (
  ‘report_id‘ INT UNSIGNED NOT NULL COMMENT ’1:1 Primary & Foreign Key to defect_reports.id’,
  ‘developer_description‘ TEXT NULL DEFAULT NULL,
  ‘classification_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL,
  ‘date_originated‘ DATE NULL DEFAULT NULL,
  ‘activity_originated‘ TEXT NULL DEFAULT NULL,
  ‘deliverable‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘what_to_do‘ TEXT NULL DEFAULT NULL,
  ‘assigned_on‘ DATE NULL DEFAULT NULL,
  ‘uniqueness‘ ENUM(’new’, ’duplicate’) NOT NULL DEFAULT ’new’,
  ‘duplicate_of_report_id‘ INT UNSIGNED NULL DEFAULT NULL,
  ‘priority‘ ENUM(’most_urgent’, ’average’, ’less_urgent’) NOT NULL DEFAULT ’average’,
  ‘severity‘ ENUM(’minor’, ’major’, ’critical’) NOT NULL DEFAULT ’minor’,
  ‘workaround‘ TEXT NULL DEFAULT NULL,
  ‘proposed_fix‘ TEXT NULL DEFAULT NULL,
  ‘status‘ ENUM(’submitted’, ’assigned’, ’postponed’, ’opened’, ’fixed’, ’resolved’, ’closed’) N
    OT NULL DEFAULT ’submitted’,
  ‘fix_description‘ TEXT NULL DEFAULT NULL,
  ‘fix_started_on‘ DATE NULL DEFAULT NULL,
  ‘fix_completed_on‘ DATE NULL DEFAULT NULL,
  ‘fix_impact‘ TEXT NULL DEFAULT NULL,
  ‘fix_effort‘ DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  ‘fix_cost‘ DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  ‘resolution_description‘ TEXT NULL DEFAULT NULL,
  ‘resolution_completed_on‘ DATE NULL DEFAULT NULL,
  ‘resolution_closed_on‘ DATE NULL DEFAULT NULL,
  ‘cost_of_defect‘ DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  ‘activity_found‘ TEXT NULL DEFAULT NULL,
  PRIMARY KEY (‘report_id‘),
  KEY ‘idx_defect_analyses_classification_id‘ (‘classification_id‘),
  KEY ‘idx_defect_analyses_duplicate_of‘ (‘duplicate_of_report_id‘),
  KEY ‘idx_defect_analyses_status_priority‘ (‘status‘, ‘priority‘, ‘severity‘),
  CONSTRAINT ‘fk_defect_analyses_report‘
    FOREIGN KEY (‘report_id‘) REFERENCES ‘defect_reports‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT ‘fk_defect_analyses_classification‘
    FOREIGN KEY (‘classification_id‘) REFERENCES ‘fault_classifications‘ (‘id‘)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT ‘fk_defect_analyses_duplicate_of‘
    FOREIGN KEY (‘duplicate_of_report_id‘) REFERENCES ‘defect_reports‘ (‘id‘)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 10. Table: ‘analysis_phases‘ (junction table for defect_analyses <-> sdlc_phases -- 1NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘analysis_phases‘ (
  ‘report_id‘ INT UNSIGNED NOT NULL,
  ‘phase_id‘ TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (‘report_id‘, ‘phase_id‘),
  KEY ‘idx_analysis_phases_phase_id‘ (‘phase_id‘),
  CONSTRAINT ‘fk_analysis_phases_analysis‘
    FOREIGN KEY (‘report_id‘) REFERENCES ‘defect_analyses‘ (‘report_id‘)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT ‘fk_analysis_phases_phase‘
    FOREIGN KEY (‘phase_id‘) REFERENCES ‘sdlc_phases‘ (‘id‘)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;
```

==========

SECTION 2.2 Zero-Data-Loss DML Migration Script (database/02_migrate_data.sql)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\02_migrate_data.sql

```
================================================================================

-- ============================================================================
-- File: 02_migrate_data.sql
-- Description: Idempotent, Zero-Data-Loss DML Migration from Legacy Tables
--              (‘product‘, ‘users‘, ‘user_sessions‘, ‘raport‘, ‘analyse‘)
--              into the 3NF Schema (‘products‘, ‘companies‘, ‘users‘,
--              ‘user_sessions‘, ‘password_resets‘, ‘sdlc_phases‘,
--              ‘fault_classifications‘, ‘defect_reports‘, ‘defect_analyses‘,
--              ‘analysis_phases‘).
-- ============================================================================

SET NAMES utf8mb4;

-- ============================================================================
-- PRE-FLIGHT DDL PREPARATION (for in-place migration on an existing database)
-- Note: MySQL implicitly commits on DDL (CREATE/ALTER/DROP/RENAME), so schema
-- preparation runs prior to the atomic DML transaction block below.
-- ============================================================================
SET FOREIGN_KEY_CHECKS = 0;

CREATE TABLE IF NOT EXISTS ‘products‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(100) NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_products_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ‘companies‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(100) NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_companies_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Add ‘company_id‘ to ‘users‘ if migrating in-place from legacy ‘users‘
SET @col_exists := (
  SELECT COUNT(*)
  FROM INFORMATION_SCHEMA.COLUMNS
  WHERE TABLE_SCHEMA = DATABASE()
    AND TABLE_NAME = ’users’
    AND COLUMN_NAME = ’company_id’
);
SET @sql_add_company_id := IF(
  @col_exists = 0,
  ’ALTER TABLE ‘users‘ ADD COLUMN ‘company_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL AFTER ‘userle
    vel‘, ADD KEY ‘idx_users_company_id‘ (‘company_id‘), ADD CONSTRAINT ‘fk_users_company‘ FOREI
    GN KEY (‘company_id‘) REFERENCES ‘companies‘ (‘id‘) ON DELETE SET NULL ON UPDATE CASCADE’,
  ’SELECT 1’
);
PREPARE stmt FROM @sql_add_company_id;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;

CREATE TABLE IF NOT EXISTS ‘password_resets‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘user_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘token_hash‘ CHAR(64) NOT NULL,
  ‘expires_at‘ DATETIME NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_password_resets_token_hash‘ (‘token_hash‘),
  KEY ‘idx_password_resets_user_expires‘ (‘user_id‘, ‘expires_at‘),
```

```
  CONSTRAINT ‘fk_password_resets_user‘
    FOREIGN KEY (‘user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ‘sdlc_phases‘ (
  ‘id‘ TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(64) NOT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_sdlc_phases_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ‘fault_classifications‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘code‘ VARCHAR(100) NOT NULL,
  ‘description‘ VARCHAR(255) NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_fault_classifications_code‘ (‘code‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ‘defect_reports‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘product_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘environment‘ TEXT NULL DEFAULT NULL,
  ‘reporter_user_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL,
  ‘guest_name‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘guest_email‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘reported_at‘ DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ‘title‘ VARCHAR(255) NOT NULL,
  ‘description‘ TEXT NOT NULL,
  ‘discovered_by‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘person_type‘ ENUM(’Customer’, ’Developer’, ’Tester’) NOT NULL DEFAULT ’Customer’,
  ‘occurred_on‘ DATE NOT NULL,
  ‘location‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘initial_severity‘ ENUM(’minor’, ’major’, ’critical’) NOT NULL DEFAULT ’minor’,
  ‘initial_priority‘ ENUM(’most_urgent’, ’average’, ’less_urgent’) NOT NULL DEFAULT ’average’,
  ‘additional_info‘ TEXT NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  KEY ‘idx_defect_reports_product_id‘ (‘product_id‘),
  KEY ‘idx_defect_reports_reporter_user_id‘ (‘reporter_user_id‘),
  KEY ‘idx_defect_reports_reported_at‘ (‘reported_at‘ DESC),
  KEY ‘idx_defect_reports_severity_reported‘ (‘initial_severity‘, ‘reported_at‘ DESC),
  KEY ‘idx_defect_reports_priority_reported‘ (‘initial_priority‘, ‘reported_at‘ DESC),
  CONSTRAINT ‘fk_defect_reports_product‘
    FOREIGN KEY (‘product_id‘) REFERENCES ‘products‘ (‘id‘)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT ‘fk_defect_reports_reporter‘
    FOREIGN KEY (‘reporter_user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT ‘chk_defect_reports_reporter_identity‘
    CHECK (
      ‘reporter_user_id‘ IS NOT NULL
      OR (‘guest_name‘ IS NOT NULL AND ‘guest_email‘ IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ‘defect_analyses‘ (
  ‘report_id‘ INT UNSIGNED NOT NULL,
  ‘developer_description‘ TEXT NULL DEFAULT NULL,
  ‘classification_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL,
  ‘date_originated‘ DATE NULL DEFAULT NULL,
  ‘activity_originated‘ TEXT NULL DEFAULT NULL,
  ‘deliverable‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘what_to_do‘ TEXT NULL DEFAULT NULL,
  ‘assigned_on‘ DATE NULL DEFAULT NULL,
```

```
  `uniqueness` ENUM('new', 'duplicate') NOT NULL DEFAULT 'new',
  `duplicate_of_report_id` INT UNSIGNED NULL DEFAULT NULL,
  `priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT 'average',
  `severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `workaround` TEXT NULL DEFAULT NULL,
  `proposed_fix` TEXT NULL DEFAULT NULL,
  `status` ENUM('submitted', 'assigned', 'postponed', 'opened', 'fixed', 'resolved', 'closed') N
    OT NULL DEFAULT 'submitted',
  `fix_description` TEXT NULL DEFAULT NULL,
  `fix_started_on` DATE NULL DEFAULT NULL,
  `fix_completed_on` DATE NULL DEFAULT NULL,
  `fix_impact` TEXT NULL DEFAULT NULL,
  `fix_effort` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  `fix_cost` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `resolution_description` TEXT NULL DEFAULT NULL,
  `resolution_completed_on` DATE NULL DEFAULT NULL,
  `resolution_closed_on` DATE NULL DEFAULT NULL,
  `cost_of_defect` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `activity_found` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`report_id`),
  KEY `idx_defect_analyses_classification_id` (`classification_id`),
  KEY `idx_defect_analyses_duplicate_of` (`duplicate_of_report_id`),
  KEY `idx_defect_analyses_status_priority` (`status`, `priority`, `severity`),
  CONSTRAINT `fk_defect_analyses_report`
    FOREIGN KEY (`report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_classification`
    FOREIGN KEY (`classification_id`) REFERENCES `fault_classifications` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_duplicate_of`
    FOREIGN KEY (`duplicate_of_report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `analysis_phases` (
  `report_id` INT UNSIGNED NOT NULL,
  `phase_id` TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (`report_id`, `phase_id`),
  KEY `idx_analysis_phases_phase_id` (`phase_id`),
  CONSTRAINT `fk_analysis_phases_analysis`
    FOREIGN KEY (`report_id`) REFERENCES `defect_analyses` (`report_id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_analysis_phases_phase`
    FOREIGN KEY (`phase_id`) REFERENCES `sdlc_phases` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;

-- ============================================================================
-- ATOMIC DML DATA MIGRATION (START TRANSACTION ... COMMIT)
-- ============================================================================
START TRANSACTION;

-- ----------------------------------------------------------------------------
-- 1. Migrate `product` -> `products` and backfill uncataloged `raport.Product`
-- ----------------------------------------------------------------------------
INSERT INTO `products` (`id`, `name`, `created_at`)
SELECT
  p.`id`,
  TRIM(p.`name`),
  p.`created_at`
FROM `product` p
ON DUPLICATE KEY UPDATE
  `name` = VALUES(`name`);
```

```
INSERT IGNORE INTO ‘products‘ (‘name‘)
SELECT DISTINCT
  COALESCE(NULLIF(TRIM(r.‘Product‘), ’’), ’Unknown Product’)
FROM ‘raport‘ r
LEFT JOIN ‘products‘ p
  ON p.‘name‘ = COALESCE(NULLIF(TRIM(r.‘Product‘), ’’), ’Unknown Product’)
WHERE p.‘id‘ IS NULL;

-- Backfill NULL ‘raport.product_id‘ by matching ‘raport.Product = products.name‘
UPDATE ‘raport‘ r
INNER JOIN ‘products‘ p
  ON p.‘name‘ = COALESCE(NULLIF(TRIM(r.‘Product‘), ’’), ’Unknown Product’)
SET r.‘product_id‘ = p.‘id‘
WHERE r.‘product_id‘ IS NULL;

-- ----------------------------------------------------------------------------
-- 2. Extract distinct non-null ‘users.company‘ into ‘companies‘ & link ‘company_id‘
-- ----------------------------------------------------------------------------
INSERT IGNORE INTO ‘companies‘ (‘name‘)
SELECT DISTINCT
  TRIM(u.‘company‘)
FROM ‘users‘ u
WHERE u.‘company‘ IS NOT NULL
  AND TRIM(u.‘company‘) <> ’’;

UPDATE ‘users‘ u
INNER JOIN ‘companies‘ c
  ON c.‘name‘ = TRIM(u.‘company‘)
SET u.‘company_id‘ = c.‘id‘
WHERE u.‘company‘ IS NOT NULL
  AND TRIM(u.‘company‘) <> ’’;

-- ----------------------------------------------------------------------------
-- 3. Migrate active non-null ‘reset_token_hash‘ rows from ‘users‘ to ‘password_resets‘
-- ----------------------------------------------------------------------------
INSERT IGNORE INTO ‘password_resets‘ (‘user_id‘, ‘token_hash‘, ‘expires_at‘, ‘created_at‘)
SELECT
  u.‘id‘,
  u.‘reset_token_hash‘,
  COALESCE(u.‘reset_token_expires_at‘, DATE_ADD(NOW(), INTERVAL 1 HOUR)),
  u.‘updated_at‘
FROM ‘users‘ u
WHERE u.‘reset_token_hash‘ IS NOT NULL
  AND TRIM(u.‘reset_token_hash‘) <> ’’
  AND (u.‘reset_token_expires_at‘ IS NULL OR u.‘reset_token_expires_at‘ > NOW());

-- ----------------------------------------------------------------------------
-- 4. Migrate ‘raport‘ -> ‘defect_reports‘
--    - Converts ‘Whenreported‘ VARCHAR(32) to DATETIME ‘reported_at‘ via STR_TO_DATE()
--    - Normalizes reporter identity (‘reporter_user_id‘ vs ‘guest_name‘/‘guest_email‘)
--    - Standardizes ‘Priority‘ (’m_urgent’ -> ’most_urgent’, ’l_urgent’ -> ’less_urgent’)
-- ----------------------------------------------------------------------------
INSERT INTO ‘defect_reports‘ (
  ‘id‘,
  ‘product_id‘,
  ‘environment‘,
  ‘reporter_user_id‘,
  ‘guest_name‘,
  ‘guest_email‘,
  ‘reported_at‘,
  ‘title‘,
  ‘description‘,
  ‘discovered_by‘,
  ‘person_type‘,
```

```
  ‘occurred_on‘,
  ‘location‘,
  ‘initial_severity‘,
  ‘initial_priority‘,
  ‘additional_info‘
)
SELECT
  r.‘ID‘ AS ‘id‘,
  COALESCE(p_id.‘id‘, p_name.‘id‘) AS ‘product_id‘,
  r.‘Environment‘ AS ‘environment‘,
  u.‘id‘ AS ‘reporter_user_id‘,
  CASE
    WHEN u.‘id‘ IS NOT NULL THEN NULL
    ELSE COALESCE(NULLIF(TRIM(r.‘Who‘), ’’), ’Anonymous Guest’)
  END AS ‘guest_name‘,
  CASE
    WHEN u.‘id‘ IS NOT NULL THEN NULL
    ELSE COALESCE(NULLIF(TRIM(r.‘Contactemail‘), ’’), ’unknown@example.invalid’)
  END AS ‘guest_email‘,
  COALESCE(
    STR_TO_DATE(TRIM(r.‘Whenreported‘), ’%Y-%m-%d %H:%i:%s’),
    STR_TO_DATE(TRIM(r.‘Whenreported‘), ’%Y-%m-%d %H:%i’),
    STR_TO_DATE(TRIM(r.‘Whenreported‘), ’%Y-%m-%d’),
    CAST(r.‘Submitted‘ AS DATETIME),
    NOW()
  ) AS ‘reported_at‘,
  r.‘Title‘ AS ‘title‘,
  r.‘Description‘ AS ‘description‘,
  r.‘Discoveredby‘ AS ‘discovered_by‘,
  r.‘Persontype‘ AS ‘person_type‘,
  r.‘Occured‘ AS ‘occurred_on‘,
  r.‘Location‘ AS ‘location‘,
  r.‘Severity‘ AS ‘initial_severity‘,
  CASE r.‘Priority‘
    WHEN ’m_urgent’ THEN ’most_urgent’
    WHEN ’average’  THEN ’average’
    WHEN ’l_urgent’ THEN ’less_urgent’
    ELSE ’average’
  END AS ‘initial_priority‘,
  r.‘Addinfo‘ AS ‘additional_info‘
FROM ‘raport‘ r
LEFT JOIN ‘products‘ p_id
  ON p_id.‘id‘ = r.‘product_id‘
LEFT JOIN ‘products‘ p_name
  ON p_name.‘name‘ = COALESCE(NULLIF(TRIM(r.‘Product‘), ’’), ’Unknown Product’)
LEFT JOIN ‘users‘ u
  ON u.‘id‘ = r.‘CustomerID‘
ON DUPLICATE KEY UPDATE
  ‘product_id‘       = VALUES(‘product_id‘),
  ‘reporter_user_id‘ = VALUES(‘reporter_user_id‘),
  ‘guest_name‘       = VALUES(‘guest_name‘),
  ‘guest_email‘      = VALUES(‘guest_email‘),
  ‘reported_at‘      = VALUES(‘reported_at‘),
  ‘initial_priority‘ = VALUES(‘initial_priority‘);

-- ----------------------------------------------------------------------------
-- 5. Migrate ‘analyse.classification‘ -> ‘fault_classifications‘
-- ----------------------------------------------------------------------------
INSERT IGNORE INTO ‘fault_classifications‘ (‘code‘)
SELECT DISTINCT
  TRIM(a.‘classification‘)
FROM ‘analyse‘ a
WHERE a.‘classification‘ IS NOT NULL
  AND TRIM(a.‘classification‘) <> ’’;
```

```
-- ------------------------------------------------------------------------
-- 6. Migrate ‘analyse‘ -> ‘defect_analyses‘
--    - Standardizes ‘priority‘ (’most urgent’ -> ’most_urgent’, ’less urgent’ -> ’less_urgent’)
--    - Standardizes ‘uniqueness‘ (’New defect’ -> ’new’, ’Already submitted’ -> ’duplicate’)
--    - Preserves legacy ‘uniqueness = ’Critical’‘ by setting ‘severity = ’critical’‘ and ‘uniqu
    eness = ’new’‘
-- ------------------------------------------------------------------------
INSERT INTO ‘defect_analyses‘ (
  ‘report_id‘,
  ‘developer_description‘,
  ‘classification_id‘,
  ‘date_originated‘,
  ‘activity_originated‘,
  ‘deliverable‘,
  ‘what_to_do‘,
  ‘assigned_on‘,
  ‘uniqueness‘,
  ‘duplicate_of_report_id‘,
  ‘priority‘,
  ‘severity‘,
  ‘workaround‘,
  ‘proposed_fix‘,
  ‘status‘,
  ‘fix_description‘,
  ‘fix_started_on‘,
  ‘fix_completed_on‘,
  ‘fix_impact‘,
  ‘fix_effort‘,
  ‘fix_cost‘,
  ‘resolution_description‘,
  ‘resolution_completed_on‘,
  ‘resolution_closed_on‘,
  ‘cost_of_defect‘,
  ‘activity_found‘
)
SELECT
  a.‘id‘ AS ‘report_id‘,
  a.‘devdesc‘ AS ‘developer_description‘,
  fc.‘id‘ AS ‘classification_id‘,
  a.‘dateoriginated‘ AS ‘date_originated‘,
  a.‘actoriginated‘ AS ‘activity_originated‘,
  a.‘deliverable‘ AS ‘deliverable‘,
  a.‘whattodo‘ AS ‘what_to_do‘,
  a.‘assigned‘ AS ‘assigned_on‘,
  CASE a.‘uniqueness‘
    WHEN ’Already submitted’ THEN ’duplicate’
    WHEN ’New defect’        THEN ’new’
    WHEN ’Critical’          THEN ’new’
    ELSE ’new’
  END AS ‘uniqueness‘,
  NULL AS ‘duplicate_of_report_id‘,
  CASE a.‘priority‘
    WHEN ’most urgent’ THEN ’most_urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less urgent’ THEN ’less_urgent’
    ELSE ’average’
  END AS ‘priority‘,
  CASE
    WHEN a.‘uniqueness‘ = ’Critical’ THEN ’critical’
    ELSE a.‘severity‘
  END AS ‘severity‘,
  a.‘workaround‘ AS ‘workaround‘,
  a.‘proposedfix‘ AS ‘proposed_fix‘,
  a.‘status‘ AS ‘status‘,
  a.‘fixdesc‘ AS ‘fix_description‘,
```

```
  a.‘fixstarted‘ AS ‘fix_started_on‘,
  a.‘fixcomplete‘ AS ‘fix_completed_on‘,
  a.‘fiximpact‘ AS ‘fix_impact‘,
  a.‘fixeffort‘ AS ‘fix_effort‘,
  a.‘fixcost‘ AS ‘fix_cost‘,
  a.‘resodesc‘ AS ‘resolution_description‘,
  a.‘resocomplete‘ AS ‘resolution_completed_on‘,
  a.‘resoclosed‘ AS ‘resolution_closed_on‘,
  a.‘costofdefect‘ AS ‘cost_of_defect‘,
  a.‘actfound‘ AS ‘activity_found‘
FROM ‘analyse‘ a
INNER JOIN ‘defect_reports‘ dr
  ON dr.‘id‘ = a.‘id‘
LEFT JOIN ‘fault_classifications‘ fc
  ON fc.‘code‘ = TRIM(a.‘classification‘)
ON DUPLICATE KEY UPDATE
  ‘classification_id‘ = VALUES(‘classification_id‘),
  ‘uniqueness‘        = VALUES(‘uniqueness‘),
  ‘priority‘          = VALUES(‘priority‘),
  ‘severity‘          = VALUES(‘severity‘);

-- ------------------------------------------------------------------------
-- 7. Split comma-delimited ‘analyse.phase‘ into ‘sdlc_phases‘ and ‘analysis_phases‘
-- ------------------------------------------------------------------------
INSERT IGNORE INTO ‘sdlc_phases‘ (‘name‘)
WITH RECURSIVE ‘phase_splitter‘ (‘report_id‘, ‘phase_token‘, ‘remainder‘) AS (
  SELECT
    a.‘id‘ AS ‘report_id‘,
    TRIM(SUBSTRING_INDEX(a.‘phase‘, ’,’, 1)) AS ‘phase_token‘,
    SUBSTRING(a.‘phase‘, CHAR_LENGTH(SUBSTRING_INDEX(a.‘phase‘, ’,’, 1)) + 2) AS ‘remainder‘
  FROM ‘analyse‘ a
  WHERE a.‘phase‘ IS NOT NULL
    AND TRIM(a.‘phase‘) <> ’’
  UNION ALL
  SELECT
    ps.‘report_id‘,
    TRIM(SUBSTRING_INDEX(ps.‘remainder‘, ’,’, 1)) AS ‘phase_token‘,
    SUBSTRING(ps.‘remainder‘, CHAR_LENGTH(SUBSTRING_INDEX(ps.‘remainder‘, ’,’, 1)) + 2) AS ‘rema
    inder‘
  FROM ‘phase_splitter‘ ps
  WHERE ps.‘remainder‘ <> ’’
)
SELECT DISTINCT
  ps.‘phase_token‘
FROM ‘phase_splitter‘ ps
WHERE ps.‘phase_token‘ <> ’’;

INSERT IGNORE INTO ‘analysis_phases‘ (‘report_id‘, ‘phase_id‘)
WITH RECURSIVE ‘phase_splitter‘ (‘report_id‘, ‘phase_token‘, ‘remainder‘) AS (
  SELECT
    a.‘id‘ AS ‘report_id‘,
    TRIM(SUBSTRING_INDEX(a.‘phase‘, ’,’, 1)) AS ‘phase_token‘,
    SUBSTRING(a.‘phase‘, CHAR_LENGTH(SUBSTRING_INDEX(a.‘phase‘, ’,’, 1)) + 2) AS ‘remainder‘
  FROM ‘analyse‘ a
  INNER JOIN ‘defect_analyses‘ da
    ON da.‘report_id‘ = a.‘id‘
  WHERE a.‘phase‘ IS NOT NULL
    AND TRIM(a.‘phase‘) <> ’’
  UNION ALL
  SELECT
    ps.‘report_id‘,
    TRIM(SUBSTRING_INDEX(ps.‘remainder‘, ’,’, 1)) AS ‘phase_token‘,
    SUBSTRING(ps.‘remainder‘, CHAR_LENGTH(SUBSTRING_INDEX(ps.‘remainder‘, ’,’, 1)) + 2) AS ‘rema
    inder‘
  FROM ‘phase_splitter‘ ps
```

```
  WHERE ps.‘remainder‘ <> ’’
)
SELECT
  ps.‘report_id‘,
  sp.‘id‘ AS ‘phase_id‘
FROM ‘phase_splitter‘ ps
INNER JOIN ‘sdlc_phases‘ sp
  ON sp.‘name‘ = ps.‘phase_token‘
WHERE ps.‘phase_token‘ <> ’’;

COMMIT;

-- ============================================================================
-- POST-MIGRATION CLEANUP (Drop denormalized columns & legacy tables)
-- ============================================================================
SET FOREIGN_KEY_CHECKS = 0;

ALTER TABLE ‘users‘
  DROP COLUMN IF EXISTS ‘company‘,
  DROP COLUMN IF EXISTS ‘logged‘,
  DROP COLUMN IF EXISTS ‘reset_token_hash‘,
  DROP COLUMN IF EXISTS ‘reset_token_expires_at‘;

DROP TABLE IF EXISTS ‘analyse‘;
DROP TABLE IF EXISTS ‘raport‘;
DROP TABLE IF EXISTS ‘product‘;

SET FOREIGN_KEY_CHECKS = 1;


================================================================================
SECTION 2.3 Backward-Compatibility Views (database/03_legacy_views.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\03_legacy_views.sql
================================================================================

-- ============================================================================
-- File: 03_legacy_views.sql
-- Description: Backward-Compatibility Read Views (‘raport‘, ‘analyse‘, ‘product‘)
--              Reconstructs exact legacy column names and formats from 3NF tables.
-- ============================================================================

SET NAMES utf8mb4;

-- ----------------------------------------------------------------------------
-- 1. View: ‘product‘ (compatibility alias for ‘products‘)
-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘product‘ AS
SELECT
  p.‘id‘,
  p.‘name‘,
  p.‘created_at‘
FROM ‘products‘ p;

-- ----------------------------------------------------------------------------
-- 2. View: ‘raport‘ (reconstructs legacy ‘raport‘ format from ‘defect_reports‘)
-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘raport‘ AS
SELECT
  dr.‘id‘ AS ‘ID‘,
  p.‘name‘ AS ‘Product‘,
  dr.‘product_id‘ AS ‘product_id‘,
  dr.‘environment‘ AS ‘Environment‘,
  dr.‘reporter_user_id‘ AS ‘CustomerID‘,
  COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘Who‘,
  COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘Contactemail‘,
```

```
  DATE_FORMAT(dr.‘reported_at‘, ’%Y-%m-%d %H:%i’) AS ‘Whenreported‘,
  dr.‘title‘ AS ‘Title‘,
  dr.‘description‘ AS ‘Description‘,
  dr.‘discovered_by‘ AS ‘Discoveredby‘,
  dr.‘person_type‘ AS ‘Persontype‘,
  dr.‘occurred_on‘ AS ‘Occured‘,
  dr.‘location‘ AS ‘Location‘,
  dr.‘initial_severity‘ AS ‘Severity‘,
  CASE dr.‘initial_priority‘
    WHEN ’most_urgent’ THEN ’m_urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’l_urgent’
  END AS ‘Priority‘,
  DATE(dr.‘reported_at‘) AS ‘Submitted‘,
  dr.‘additional_info‘ AS ‘Addinfo‘
FROM ‘defect_reports‘ dr
INNER JOIN ‘products‘ p
  ON p.‘id‘ = dr.‘product_id‘
LEFT JOIN ‘users‘ u
  ON u.‘id‘ = dr.‘reporter_user_id‘;

-- ------------------------------------------------------------------------------
-- 3. View: ‘analyse‘ (reconstructs legacy ‘analyse‘ format from ‘defect_analyses‘)
-- ------------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘analyse‘ AS
SELECT
  da.‘report_id‘ AS ‘id‘,
  da.‘developer_description‘ AS ‘devdesc‘,
  fc.‘code‘ AS ‘classification‘,
  da.‘date_originated‘ AS ‘dateoriginated‘,
  ph.‘phases_csv‘ AS ‘phase‘,
  da.‘activity_originated‘ AS ‘actoriginated‘,
  da.‘deliverable‘ AS ‘deliverable‘,
  da.‘what_to_do‘ AS ‘whattodo‘,
  DATE(dr.‘reported_at‘) AS ‘submitted‘,
  da.‘assigned_on‘ AS ‘assigned‘,
  CASE da.‘uniqueness‘
    WHEN ’duplicate’ THEN ’Already submitted’
    WHEN ’new’       THEN ’New defect’
  END AS ‘uniqueness‘,
  CASE da.‘priority‘
    WHEN ’most_urgent’ THEN ’most urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’less urgent’
  END AS ‘priority‘,
  da.‘severity‘ AS ‘severity‘,
  da.‘workaround‘ AS ‘workaround‘,
  da.‘proposed_fix‘ AS ‘proposedfix‘,
  da.‘status‘ AS ‘status‘,
  da.‘fix_description‘ AS ‘fixdesc‘,
  da.‘fix_started_on‘ AS ‘fixstarted‘,
  da.‘fix_completed_on‘ AS ‘fixcomplete‘,
  da.‘fix_impact‘ AS ‘fiximpact‘,
  da.‘fix_effort‘ AS ‘fixeffort‘,
  da.‘fix_cost‘ AS ‘fixcost‘,
  da.‘resolution_description‘ AS ‘resodesc‘,
  da.‘resolution_completed_on‘ AS ‘resocomplete‘,
  da.‘resolution_closed_on‘ AS ‘resoclosed‘,
  da.‘cost_of_defect‘ AS ‘costofdefect‘,
  da.‘activity_found‘ AS ‘actfound‘
FROM ‘defect_analyses‘ da
INNER JOIN ‘defect_reports‘ dr
  ON dr.‘id‘ = da.‘report_id‘
LEFT JOIN ‘fault_classifications‘ fc
  ON fc.‘id‘ = da.‘classification_id‘
```

```
LEFT JOIN (
  SELECT
    ap.‘report_id‘,
    GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
  FROM ‘analysis_phases‘ ap
  INNER JOIN ‘sdlc_phases‘ sp
    ON sp.‘id‘ = ap.‘phase_id‘
  GROUP BY ap.‘report_id‘
) ph
  ON ph.‘report_id‘ = da.‘report_id‘;


================================================================================
SECTION 2.4 Updated Mermaid ER Diagram (database/04_er_diagram.mmd)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\04_er_diagram.mmd
================================================================================

erDiagram
    companies ||--o{ users : "employs (ON DELETE SET NULL)"
    users ||--o{ user_sessions : "authenticates (ON DELETE CASCADE)"
    users ||--o{ password_resets : "requests (ON DELETE CASCADE)"
    users ||--o{ defect_reports : "reports (ON DELETE SET NULL)"
    products ||--o{ defect_reports : "categorizes (ON DELETE RESTRICT)"
    defect_reports ||--o| defect_analyses : "is analyzed by (1:1 ON DELETE CASCADE)"
    defect_reports ||--o{ defect_analyses : "is duplicated by (ON DELETE SET NULL)"
    fault_classifications ||--o{ defect_analyses : "classifies (ON DELETE SET NULL)"
    defect_analyses ||--o{ analysis_phases : "spans (ON DELETE CASCADE)"
    sdlc_phases ||--o{ analysis_phases : "defines (ON DELETE RESTRICT)"

    products {
        SMALLINT_UNSIGNED id PK "AUTO_INCREMENT"
        VARCHAR_100 name UK "NOT NULL (uk_products_name)"
        TIMESTAMP created_at "NOT NULL DEFAULT CURRENT_TIMESTAMP"
    }

    companies {
        SMALLINT_UNSIGNED id PK "AUTO_INCREMENT"
        VARCHAR_100 name UK "NOT NULL (uk_companies_name)"
        TIMESTAMP created_at "NOT NULL DEFAULT CURRENT_TIMESTAMP"
    }

    users {
        SMALLINT_UNSIGNED id PK "AUTO_INCREMENT"
        VARCHAR_64 login UK "NOT NULL (uk_users_login)"
        VARCHAR_255 password "NOT NULL (Argon2id hash)"
        VARCHAR_255 email UK "NOT NULL (uk_users_email)"
        TINYINT_UNSIGNED userlevel "NOT NULL DEFAULT 0 (0=Applying..4=Admin)"
        SMALLINT_UNSIGNED company_id FK "NULL -> companies.id"
        VARCHAR_50 phone "NULL"
        VARCHAR_100 name "NULL"
        TINYINT_1 activate "NOT NULL DEFAULT 0"
        TIMESTAMP created_at "NOT NULL DEFAULT CURRENT_TIMESTAMP"
        TIMESTAMP updated_at "NOT NULL ON UPDATE CURRENT_TIMESTAMP"
    }

    user_sessions {
        BIGINT_UNSIGNED id PK "AUTO_INCREMENT"
        SMALLINT_UNSIGNED user_id FK "NOT NULL -> users.id"
        CHAR_64 token_hash UK "NOT NULL (uk_user_sessions_token_hash)"
        VARCHAR_45 ip_address "NULL"
        VARCHAR_255 user_agent "NULL"
        DATETIME expires_at "NOT NULL"
        TIMESTAMP created_at "NOT NULL DEFAULT CURRENT_TIMESTAMP"
    }
```

```
    password_resets {
        INT_UNSIGNED id PK "AUTO_INCREMENT"
        SMALLINT_UNSIGNED user_id FK "NOT NULL -> users.id"
        CHAR_64 token_hash UK "NOT NULL (uk_password_resets_token_hash)"
        DATETIME expires_at "NOT NULL"
        TIMESTAMP created_at "NOT NULL DEFAULT CURRENT_TIMESTAMP"
    }

    sdlc_phases {
        TINYINT_UNSIGNED id PK "AUTO_INCREMENT"
        VARCHAR_64 name UK "NOT NULL (uk_sdlc_phases_name)"
    }

    fault_classifications {
        SMALLINT_UNSIGNED id PK "AUTO_INCREMENT"
        VARCHAR_100 code UK "NOT NULL (IEEE 1044-1992)"
        VARCHAR_255 description "NULL"
    }

    defect_reports {
        INT_UNSIGNED id PK "AUTO_INCREMENT"
        SMALLINT_UNSIGNED product_id FK "NOT NULL -> products.id"
        TEXT environment "NULL"
        SMALLINT_UNSIGNED reporter_user_id FK "NULL -> users.id"
        VARCHAR_100 guest_name "NULL"
        VARCHAR_255 guest_email "NULL"
        DATETIME reported_at "NOT NULL DEFAULT CURRENT_TIMESTAMP"
        VARCHAR_255 title "NOT NULL"
        TEXT description "NOT NULL"
        VARCHAR_100 discovered_by "NULL"
        ENUM person_type "NOT NULL ('Customer','Developer','Tester')"
        DATE occurred_on "NOT NULL"
        VARCHAR_255 location "NULL"
        ENUM initial_severity "NOT NULL ('minor','major','critical')"
        ENUM initial_priority "NOT NULL ('most_urgent','average','less_urgent')"
        TEXT additional_info "NULL"
    }

    defect_analyses {
        INT_UNSIGNED report_id PK,FK "NOT NULL -> defect_reports.id"
        TEXT developer_description "NULL"
        SMALLINT_UNSIGNED classification_id FK "NULL -> fault_classifications.id"
        DATE date_originated "NULL"
        TEXT activity_originated "NULL"
        VARCHAR_255 deliverable "NULL"
        TEXT what_to_do "NULL"
        DATE assigned_on "NULL"
        ENUM uniqueness "NOT NULL ('new','duplicate')"
        INT_UNSIGNED duplicate_of_report_id FK "NULL -> defect_reports.id"
        ENUM priority "NOT NULL ('most_urgent','average','less_urgent')"
        ENUM severity "NOT NULL ('minor','major','critical')"
        TEXT workaround "NULL"
        TEXT proposed_fix "NULL"
        ENUM status "NOT NULL ('submitted'..'closed')"
        TEXT fix_description "NULL"
        DATE fix_started_on "NULL"
        DATE fix_completed_on "NULL"
        TEXT fix_impact "NULL"
        DECIMAL_8_2_UNSIGNED fix_effort "NULL (hours)"
        DECIMAL_10_2_UNSIGNED fix_cost "NULL (EUR)"
        TEXT resolution_description "NULL"
        DATE resolution_completed_on "NULL"
        DATE resolution_closed_on "NULL"
        DECIMAL_10_2_UNSIGNED cost_of_defect "NULL (EUR)"
        TEXT activity_found "NULL"
```

```
    }

    analysis_phases {
        INT_UNSIGNED report_id PK,FK "NOT NULL -> defect_analyses.report_id"
        TINYINT_UNSIGNED phase_id PK,FK "NOT NULL -> sdlc_phases.id"
    }
```

==========

SECTION 2.5 Unified Docker Initialization Schema (database/schema.sql)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\schema.sql

==========

```
-- ========================================================================
-- DefMan (Defect Manager) -- Modernized MySQL 8.0+ 3NF DDL Schema
-- Normalized to Third Normal Form (3NF) with Backward-Compatibility Views
-- Engine: InnoDB | Charset: utf8mb4 | Collation: utf8mb4_unicode_ci
-- ========================================================================

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE = 'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_E
    NGINE_SUBSTITUTION';

-- Drop legacy views/tables if re-initializing
DROP VIEW IF EXISTS `analyse`;
DROP VIEW IF EXISTS `raport`;
DROP VIEW IF EXISTS `product`;

DROP TABLE IF EXISTS `analysis_phases`;
DROP TABLE IF EXISTS `defect_analyses`;
DROP TABLE IF EXISTS `defect_reports`;
DROP TABLE IF EXISTS `fault_classifications`;
DROP TABLE IF EXISTS `sdlc_phases`;
DROP TABLE IF EXISTS `password_resets`;
DROP TABLE IF EXISTS `user_sessions`;
DROP TABLE IF EXISTS `users`;
DROP TABLE IF EXISTS `companies`;
DROP TABLE IF EXISTS `products`;

-- ------------------------------------------------------------------------
-- 1. Table: `products`
-- ------------------------------------------------------------------------
CREATE TABLE `products` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_products_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 2. Table: `companies` (3NF lookup table extracted from `users`)
-- ------------------------------------------------------------------------
CREATE TABLE `companies` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_companies_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 3. Table: `users`
-- User Levels:
```

```
--   0 = Applying (Pending Admin Approval - form_apply.php)
--   1 = Public   (Unauthenticated / Public reporter - form_raportpub.php)
--   2 = Customer (Authenticated defect reporter - form_raport.php)
--   3 = Analyzer (Developer / QA triage & IEEE fault cost tracking - form_analyze.php)
--   4 = Admin    (Full system access, user approval & product CRUD - form_usermanagement.php)
-- ------------------------------------------------------------------------
CREATE TABLE `users` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `login` VARCHAR(64) NOT NULL COMMENT 'Legacy max 6 chars in 2004 UI; widened for modern auth',
  `password` VARCHAR(255) NOT NULL COMMENT 'Argon2id password hash (PASSWORD_ARGON2ID)',
  `email` VARCHAR(255) NOT NULL,
  `userlevel` TINYINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '0=Applying, 1=Public, 2=Customer, 3=A
    nalyzer, 4=Admin',
  `company_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `phone` VARCHAR(50) NULL DEFAULT NULL,
  `name` VARCHAR(100) NULL DEFAULT NULL,
  `activate` TINYINT(1) NOT NULL DEFAULT 0 COMMENT '0=Pending/Inactive, 1=Active',
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_users_login` (`login`),
  UNIQUE KEY `uk_users_email` (`email`),
  KEY `idx_users_company_id` (`company_id`),
  KEY `idx_users_level_activate` (`userlevel`, `activate`),
  CONSTRAINT `fk_users_company`
    FOREIGN KEY (`company_id`) REFERENCES `companies` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 4. Table: `user_sessions` (Cryptographically secure bearer/session tokens)
-- ------------------------------------------------------------------------
CREATE TABLE `user_sessions` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` SMALLINT UNSIGNED NOT NULL,
  `token_hash` CHAR(64) NOT NULL COMMENT 'SHA-256 hash of 256-bit random_bytes token',
  `ip_address` VARCHAR(45) NULL DEFAULT NULL,
  `user_agent` VARCHAR(255) NULL DEFAULT NULL,
  `expires_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_user_sessions_token_hash` (`token_hash`),
  KEY `idx_user_sessions_user_expires` (`user_id`, `expires_at`),
  KEY `idx_user_sessions_expires_at` (`expires_at`),
  CONSTRAINT `fk_user_sessions_user`
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. Table: `password_resets` (Decoupled password recovery tokens -- 3NF)
-- ------------------------------------------------------------------------
CREATE TABLE `password_resets` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` SMALLINT UNSIGNED NOT NULL,
  `token_hash` CHAR(64) NOT NULL COMMENT 'SHA-256 hash of random_bytes(32) recovery token',
  `expires_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_password_resets_token_hash` (`token_hash`),
  KEY `idx_password_resets_user_expires` (`user_id`, `expires_at`),
  CONSTRAINT `fk_password_resets_user`
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- ------------------------------------------------------------------------
-- 6. Table: ‘sdlc_phases‘ (1NF lookup table for SDLC origin phases)
-- ------------------------------------------------------------------------
CREATE TABLE ‘sdlc_phases‘ (
  ‘id‘ TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(64) NOT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_sdlc_phases_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 7. Table: ‘fault_classifications‘ (3NF lookup table for IEEE 1044-1992 fault types)
-- ------------------------------------------------------------------------
CREATE TABLE ‘fault_classifications‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘code‘ VARCHAR(100) NOT NULL,
  ‘description‘ VARCHAR(255) NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_fault_classifications_code‘ (‘code‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 8. Table: ‘defect_reports‘ (3NF normalized defect reports)
-- ------------------------------------------------------------------------
CREATE TABLE ‘defect_reports‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘product_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘environment‘ TEXT NULL DEFAULT NULL,
  ‘reporter_user_id‘ SMALLINT UNSIGNED NULL DEFAULT NULL,
  ‘guest_name‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘guest_email‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘reported_at‘ DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ‘title‘ VARCHAR(255) NOT NULL,
  ‘description‘ TEXT NOT NULL,
  ‘discovered_by‘ VARCHAR(100) NULL DEFAULT NULL,
  ‘person_type‘ ENUM(’Customer’, ’Developer’, ’Tester’) NOT NULL DEFAULT ’Customer’,
  ‘occurred_on‘ DATE NOT NULL,
  ‘location‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘initial_severity‘ ENUM(’minor’, ’major’, ’critical’) NOT NULL DEFAULT ’minor’,
  ‘initial_priority‘ ENUM(’most_urgent’, ’average’, ’less_urgent’) NOT NULL DEFAULT ’average’,
  ‘additional_info‘ TEXT NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  KEY ‘idx_defect_reports_product_id‘ (‘product_id‘),
  KEY ‘idx_defect_reports_reporter_user_id‘ (‘reporter_user_id‘),
  KEY ‘idx_defect_reports_reported_at‘ (‘reported_at‘ DESC),
  KEY ‘idx_defect_reports_severity_reported‘ (‘initial_severity‘, ‘reported_at‘ DESC),
  KEY ‘idx_defect_reports_priority_reported‘ (‘initial_priority‘, ‘reported_at‘ DESC),
  CONSTRAINT ‘fk_defect_reports_product‘
    FOREIGN KEY (‘product_id‘) REFERENCES ‘products‘ (‘id‘)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT ‘fk_defect_reports_reporter‘
    FOREIGN KEY (‘reporter_user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT ‘chk_defect_reports_reporter_identity‘
    CHECK (
      ‘reporter_user_id‘ IS NOT NULL
      OR (‘guest_name‘ IS NOT NULL AND ‘guest_email‘ IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 9. Table: ‘defect_analyses‘ (3NF normalized 1:1 extension of ‘defect_reports‘)
-- ------------------------------------------------------------------------
CREATE TABLE ‘defect_analyses‘ (
```

```
  `report_id` INT UNSIGNED NOT NULL COMMENT '1:1 Primary & Foreign Key to defect_reports.id',
  `developer_description` TEXT NULL DEFAULT NULL,
  `classification_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `date_originated` DATE NULL DEFAULT NULL,
  `activity_originated` TEXT NULL DEFAULT NULL,
  `deliverable` VARCHAR(255) NULL DEFAULT NULL,
  `what_to_do` TEXT NULL DEFAULT NULL,
  `assigned_on` DATE NULL DEFAULT NULL,
  `uniqueness` ENUM('new', 'duplicate') NOT NULL DEFAULT 'new',
  `duplicate_of_report_id` INT UNSIGNED NULL DEFAULT NULL,
  `priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT 'average',
  `severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `workaround` TEXT NULL DEFAULT NULL,
  `proposed_fix` TEXT NULL DEFAULT NULL,
  `status` ENUM('submitted', 'assigned', 'postponed', 'opened', 'fixed', 'resolved', 'closed') N
    OT NULL DEFAULT 'submitted',
  `fix_description` TEXT NULL DEFAULT NULL,
  `fix_started_on` DATE NULL DEFAULT NULL,
  `fix_completed_on` DATE NULL DEFAULT NULL,
  `fix_impact` TEXT NULL DEFAULT NULL,
  `fix_effort` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  `fix_cost` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `resolution_description` TEXT NULL DEFAULT NULL,
  `resolution_completed_on` DATE NULL DEFAULT NULL,
  `resolution_closed_on` DATE NULL DEFAULT NULL,
  `cost_of_defect` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `activity_found` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`report_id`),
  KEY `idx_defect_analyses_classification_id` (`classification_id`),
  KEY `idx_defect_analyses_duplicate_of` (`duplicate_of_report_id`),
  KEY `idx_defect_analyses_status_priority` (`status`, `priority`, `severity`),
  CONSTRAINT `fk_defect_analyses_report`
    FOREIGN KEY (`report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_classification`
    FOREIGN KEY (`classification_id`) REFERENCES `fault_classifications` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_duplicate_of`
    FOREIGN KEY (`duplicate_of_report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 10. Table: `analysis_phases` (1NF junction table for defect_analyses <-> sdlc_phases)
-- ------------------------------------------------------------------------
CREATE TABLE `analysis_phases` (
  `report_id` INT UNSIGNED NOT NULL,
  `phase_id` TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (`report_id`, `phase_id`),
  KEY `idx_analysis_phases_phase_id` (`phase_id`),
  CONSTRAINT `fk_analysis_phases_analysis`
    FOREIGN KEY (`report_id`) REFERENCES `defect_analyses` (`report_id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_analysis_phases_phase`
    FOREIGN KEY (`phase_id`) REFERENCES `sdlc_phases` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- Backward-Compatibility Views (`product`, `raport`, `analyse`)
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW `product` AS
SELECT
  p.`id`,
  p.`name`,
```

```
  p.‘created_at‘
FROM ‘products‘ p;

CREATE OR REPLACE VIEW ‘raport‘ AS
SELECT
  dr.‘id‘ AS ‘ID‘,
  p.‘name‘ AS ‘Product‘,
  dr.‘product_id‘ AS ‘product_id‘,
  dr.‘environment‘ AS ‘Environment‘,
  dr.‘reporter_user_id‘ AS ‘CustomerID‘,
  COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘Who‘,
  COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘Contactemail‘,
  DATE_FORMAT(dr.‘reported_at‘, ’%Y-%m-%d %H:%i’) AS ‘Whenreported‘,
  dr.‘title‘ AS ‘Title‘,
  dr.‘description‘ AS ‘Description‘,
  dr.‘discovered_by‘ AS ‘Discoveredby‘,
  dr.‘person_type‘ AS ‘Persontype‘,
  dr.‘occurred_on‘ AS ‘Occured‘,
  dr.‘location‘ AS ‘Location‘,
  dr.‘initial_severity‘ AS ‘Severity‘,
  CASE dr.‘initial_priority‘
    WHEN ’most_urgent’ THEN ’m_urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’l_urgent’
  END AS ‘Priority‘,
  DATE(dr.‘reported_at‘) AS ‘Submitted‘,
  dr.‘additional_info‘ AS ‘Addinfo‘
FROM ‘defect_reports‘ dr
INNER JOIN ‘products‘ p
  ON p.‘id‘ = dr.‘product_id‘
LEFT JOIN ‘users‘ u
  ON u.‘id‘ = dr.‘reporter_user_id‘;

CREATE OR REPLACE VIEW ‘analyse‘ AS
SELECT
  da.‘report_id‘ AS ‘id‘,
  da.‘developer_description‘ AS ‘devdesc‘,
  fc.‘code‘ AS ‘classification‘,
  da.‘date_originated‘ AS ‘dateoriginated‘,
  ph.‘phases_csv‘ AS ‘phase‘,
  da.‘activity_originated‘ AS ‘actoriginated‘,
  da.‘deliverable‘ AS ‘deliverable‘,
  da.‘what_to_do‘ AS ‘whattodo‘,
  DATE(dr.‘reported_at‘) AS ‘submitted‘,
  da.‘assigned_on‘ AS ‘assigned‘,
  CASE da.‘uniqueness‘
    WHEN ’duplicate’ THEN ’Already submitted’
    WHEN ’new’       THEN ’New defect’
  END AS ‘uniqueness‘,
  CASE da.‘priority‘
    WHEN ’most_urgent’ THEN ’most urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’less urgent’
  END AS ‘priority‘,
  da.‘severity‘ AS ‘severity‘,
  da.‘workaround‘ AS ‘workaround‘,
  da.‘proposed_fix‘ AS ‘proposedfix‘,
  da.‘status‘ AS ‘status‘,
  da.‘fix_description‘ AS ‘fixdesc‘,
  da.‘fix_started_on‘ AS ‘fixstarted‘,
  da.‘fix_completed_on‘ AS ‘fixcomplete‘,
  da.‘fix_impact‘ AS ‘fiximpact‘,
  da.‘fix_effort‘ AS ‘fixeffort‘,
  da.‘fix_cost‘ AS ‘fixcost‘,
  da.‘resolution_description‘ AS ‘resodesc‘,
```

```
  da.‘resolution_completed_on‘ AS ‘resocomplete‘,
  da.‘resolution_closed_on‘ AS ‘resoclosed‘,
  da.‘cost_of_defect‘ AS ‘costofdefect‘,
  da.‘activity_found‘ AS ‘actfound‘
FROM ‘defect_analyses‘ da
INNER JOIN ‘defect_reports‘ dr
  ON dr.‘id‘ = da.‘report_id‘
LEFT JOIN ‘fault_classifications‘ fc
  ON fc.‘id‘ = da.‘classification_id‘
LEFT JOIN (
  SELECT
    ap.‘report_id‘,
    GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
  FROM ‘analysis_phases‘ ap
  INNER JOIN ‘sdlc_phases‘ sp
    ON sp.‘id‘ = ap.‘phase_id‘
  GROUP BY ap.‘report_id‘
) ph
  ON ph.‘report_id‘ = da.‘report_id‘;

SET FOREIGN_KEY_CHECKS = 1;


================================================================================
SECTION 3.1 Normalized Domain Enums (backend/src/Domain/Enum/DomainEnums.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Domain\Enum\DomainEnums.php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Domain\Enum;

/**
 * RBAC User Levels from menubar.php, form_usermanagement.php, form_apply.php
 */
enum UserLevel: int
{
    case APPLYING = 0; // Unapproved applicant (form_apply.php)
    case PUBLIC   = 1; // Public defect reporter (form_raportpub.php)
    case CUSTOMER = 2; // Authenticated customer (form_raport.php)
    case ANALYZER = 3; // Developer / QA triage & cost tracking (form_analyze.php)
    case ADMIN    = 4; // Full administrator (form_usermanagement.php, form_productdetails.php)

    public function label(): string
    {
        return match ($this) {
            self::APPLYING => ’Applying’,
            self::PUBLIC   => ’Public’,
            self::CUSTOMER => ’Customer’,
            self::ANALYZER => ’Analyzer’,
            self::ADMIN    => ’Admin’,
        };
    }
}

/**
 * Defect Severity from form_raport.php & form_analyze.php
 */
enum DefectSeverity: string
{
    case MINOR    = ’minor’;
    case MAJOR    = ’major’;
    case CRITICAL = ’critical’;
}
```

```
/**
 * Defect Priority normalized across ‘defect_reports.initial_priority‘ and ‘defect_analyses.prio
   rity‘
 * Supports 3NF DB enum (’most_urgent’, ’average’, ’less_urgent’) as well as legacy UI values
 */
enum DefectPriority: string
{
    case MOST_URGENT = ’m_urgent’;
    case AVERAGE     = ’average’;
    case LESS_URGENT = ’l_urgent’;

    public function dbValue(): string
    {
        return match ($this) {
            self::MOST_URGENT => ’most_urgent’,
            self::AVERAGE     => ’average’,
            self::LESS_URGENT => ’less_urgent’,
        };
    }

    public function analysisLabel(): string
    {
        return match ($this) {
            self::MOST_URGENT => ’most urgent’,
            self::AVERAGE     => ’average’,
            self::LESS_URGENT => ’less urgent’,
        };
    }

    public static function fromInput(string $input): self
    {
        return match (strtolower(trim($input))) {
            ’most_urgent’, ’m_urgent’, ’most urgent’ => self::MOST_URGENT,
            ’less_urgent’, ’l_urgent’, ’less urgent’ => self::LESS_URGENT,
            default                                  => self::AVERAGE,
        };
    }
}

/**
 * Defect Status from form_analyze.php & PixiJS v8 Triage Lanes
 */
enum DefectStatus: string
{
    case SUBMITTED = ’submitted’;
    case ASSIGNED  = ’assigned’;
    case POSTPONED = ’postponed’;
    case OPENED    = ’opened’;
    case FIXED     = ’fixed’;
    case RESOLVED  = ’resolved’;
    case CLOSED    = ’closed’;
}

/**
 * Person Type who discovered the defect (form_raport.php)
 */
enum PersonType: string
{
    case CUSTOMER  = ’Customer’;
    case DEVELOPER = ’Developer’;
    case TESTER    = ’Tester’;
}

/**
```

```
 * Defect Uniqueness (form_analyze.php)
 * Normalized 3NF column uses ENUM('new', 'duplicate')
 */
enum Uniqueness: string
{
    case NEW_DEFECT        = 'New defect';
    case ALREADY_SUBMITTED = 'Already submitted';
    case CRITICAL          = 'Critical';

    public function dbValue(): string
    {
        return match ($this) {
            self::ALREADY_SUBMITTED => 'duplicate',
            self::NEW_DEFECT, self::CRITICAL => 'new',
        };
    }

    public static function fromInput(string $input): self
    {
        return match (strtolower(trim($input))) {
            'duplicate', 'already submitted' => self::ALREADY_SUBMITTED,
            'critical'                       => self::CRITICAL,
            default                          => self::NEW_DEFECT,
        };
    }
}

/**
 * Lifecycle Phase in which the defect originated ('sdlc_phases')
 */
enum OriginPhase: string
{
    case REQUIREMENTS   = 'Requirements';
    case DESIGN         = 'Design';
    case IMPLEMENTATION = 'Implementation';
    case TESTING        = 'Testing';
    case DEPLOYMENT     = 'Deployment';
    case MAINTENANCE    = 'Maintenance';
}


================================================================================
SECTION 3.2 3NF Database Seeder (backend/src/Database/DatabaseSeeder.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Database\DatabaseSeeder.php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Database;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\DefectStatus;
use DefMan\Domain\Enum\IeeeFaultType;
use DefMan\Domain\Enum\OriginPhase;
use DefMan\Domain\Enum\PersonType;
use DefMan\Domain\Enum\Uniqueness;
use DefMan\Domain\Enum\UserLevel;
use PDO;

/**
 * PDO Database Seeder (3NF Normalized Schema)
 * Refactored from: install/installdb.php, install/install-part2.php, helpanalyze.html, helprepo
    rt.html
```

```
 *
 * Test/Seed Account Rule:
 * - Admin / System placeholders: zuckerberg@fb.com, billg@microsoft.com
 * - Customer / Analyzer accounts: Sabrina (sabrina@hotmail.com), Julia (julia@hotmail.com)
 */
final readonly class DatabaseSeeder
{
    public function __construct(private PDO $pdo) {}

    public function run(): void
    {
        $this->pdo->beginTransaction();

        try {
            // 1. Seed Products (‘products‘)
            $products = [
                ’MMF 1.5’,
                ’DefMan 2004 Core’,
                ’Katiska Auth Gateway 2.0’,
                ’Telemetry Engine v4’,
            ];

            $productStmt = $this->pdo->prepare(
                ’INSERT INTO ‘products‘ (‘name‘) VALUES (:name)
                 ON DUPLICATE KEY UPDATE ‘name‘ = VALUES(‘name‘)’
            );
            foreach ($products as $productName) {
                $productStmt->execute([’:name’ => $productName]);
            }

            $productMap = [];
            $rows = $this->pdo->query(’SELECT ‘id‘, ‘name‘ FROM ‘products‘’)->fetchAll();
            foreach ($rows as $row) {
                $productMap[$row[’name’]] = (int) $row[’id’];
            }

            // 2. Seed SDLC Phases (‘sdlc_phases‘ -- 1NF lookup table)
            $phaseStmt = $this->pdo->prepare(
                ’INSERT INTO ‘sdlc_phases‘ (‘name‘) VALUES (:name)
                 ON DUPLICATE KEY UPDATE ‘name‘ = VALUES(‘name‘)’
            );
            foreach (OriginPhase::cases() as $phaseCase) {
                $phaseStmt->execute([’:name’ => $phaseCase->value]);
            }

            $phaseMap = [];
            $phaseRows = $this->pdo->query(’SELECT ‘id‘, ‘name‘ FROM ‘sdlc_phases‘’)->fetchAll()
    ;
            foreach ($phaseRows as $row) {
                $phaseMap[$row[’name’]] = (int) $row[’id’];
            }

            // 3. Seed IEEE 1044-1992 Fault Classifications (‘fault_classifications‘ -- 3NF looku
    p table)
            $classificationStmt = $this->pdo->prepare(
                ’INSERT INTO ‘fault_classifications‘ (‘code‘, ‘description‘)
                 VALUES (:code, :description)
                 ON DUPLICATE KEY UPDATE ‘description‘ = VALUES(‘description‘)’
            );
            foreach (IeeeFaultType::cases() as $faultCase) {
                $subcategories = $faultCase->subcategories();
                $classificationStmt->execute([
                    ’:code’        => $faultCase->value,
                    ’:description’ => $subcategories[0] ?? null,
                ]);
```

```
        }

        $classificationMap = [];
        $classRows = $this->pdo->query('SELECT `id`, `code` FROM `fault_classifications`')->
    fetchAll();
        foreach ($classRows as $row) {
            $classificationMap[$row['code']] = (int) $row['id'];
        }

        // 4. Seed Companies (`companies` -- 3NF lookup table) & Users (`users`)
        $users = [
            [
                'login'     => 'zuck',
                'password'  => password_hash('Admin#2026!Zuck', PASSWORD_ARGON2ID),
                'email'     => 'zuckerberg@fb.com',
                'userlevel' => UserLevel::ADMIN->value,
                'company'   => 'Meta Platforms',
                'phone'     => '+1-650-543-4800',
                'name'      => 'Mark Zuckerberg',
                'activate'  => 1,
                'online'    => true,
            ],
            [
                'login'     => 'billg',
                'password'  => password_hash('System#2026!Bill', PASSWORD_ARGON2ID),
                'email'     => 'billg@microsoft.com',
                'userlevel' => UserLevel::ADMIN->value,
                'company'   => 'Microsoft Corp',
                'phone'     => '+1-425-882-8080',
                'name'      => 'Bill Gates',
                'activate'  => 1,
                'online'    => false,
            ],
            [
                'login'     => 'julia',
                'password'  => password_hash('Analyze#2026!Julia', PASSWORD_ARGON2ID),
                'email'     => 'julia@hotmail.com',
                'userlevel' => UserLevel::ANALYZER->value,
                'company'   => 'DefMan QA Labs',
                'phone'     => '+358-40-555-0103',
                'name'      => 'Julia',
                'activate'  => 1,
                'online'    => true,
            ],
            [
                'login'     => 'sabrin',
                'password'  => password_hash('Cust#2026!Sabrina', PASSWORD_ARGON2ID),
                'email'     => 'sabrina@hotmail.com',
                'userlevel' => UserLevel::CUSTOMER->value,
                'company'   => 'Nordic Enterprise Oy',
                'phone'     => '+358-40-555-0102',
                'name'      => 'Sabrina',
                'activate'  => 1,
                'online'    => false,
            ],
        ];

        $companyStmt = $this->pdo->prepare(
            'INSERT INTO `companies` (`name`) VALUES (:name)
             ON DUPLICATE KEY UPDATE `id` = LAST_INSERT_ID(`id`)'
        );

        $userStmt = $this->pdo->prepare(
            'INSERT INTO `users` (`login`, `password`, `email`, `userlevel`, `company_id`, `
    phone`, `name`, `activate`)
```

```
            VALUES (:login, :password, :email, :userlevel, :company_id, :phone, :name, :act
ivate)
            ON DUPLICATE KEY UPDATE
              `id`         = LAST_INSERT_ID(`id`),
              `userlevel`  = VALUES(`userlevel`),
              `company_id` = VALUES(`company_id`),
              `name`       = VALUES(`name`),
              `activate`   = VALUES(`activate`)'
        );

        $sessionStmt = $this->pdo->prepare(
            'INSERT IGNORE INTO `user_sessions` (`user_id`, `token_hash`, `ip_address`, `use
r_agent`, `expires_at`)
            VALUES (:user_id, :token_hash, :ip_address, :user_agent, :expires_at)'
        );

        foreach ($users as $u) {
            $companyStmt->execute([':name' => $u['company']]);
            $companyId = (int) $this->pdo->lastInsertId();

            $userStmt->execute([
                ':login'      => $u['login'],
                ':password'   => $u['password'],
                ':email'      => $u['email'],
                ':userlevel'  => $u['userlevel'],
                ':company_id' => $companyId,
                ':phone'      => $u['phone'],
                ':name'       => $u['name'],
                ':activate'   => $u['activate'],
            ]);

            $userId = (int) $this->pdo->lastInsertId();

            if ($u['online']) {
                $sessionStmt->execute([
                    ':user_id'    => $userId,
                    ':token_hash' => hash('sha256', 'seed-session-' . $u['login']),
                    ':ip_address' => '127.0.0.1',
                    ':user_agent' => 'DefMan-2026-Seed-Session/1.0',
                    ':expires_at' => (new \DateTimeImmutable('+7 days'))->format('Y-m-d H:i:
s'),
                ]);
            }
        }

        $sabrinaId = (int) $this->pdo->query(
            "SELECT `id` FROM `users` WHERE `email` = 'sabrina@hotmail.com' LIMIT 1"
        )->fetchColumn();

        // 5. Seed Sample Defects & Analyses (`defect_reports`, `defect_analyses`, `analysis
_phases`)
        $sampleDefects = [
            [
                'product_id'       => $productMap['MMF 1.5'],
                'environment'      => 'AMD Athlon XP 2000, WinXP',
                'reporter_user_id' => $sabrinaId,
                'guest_name'       => null,
                'guest_email'      => null,
                'reported_at'      => '2004-04-14 14:20:00',
                'title'            => 'PopUp Menu Crash',
                'description'      => 'When adding entries into the popup menu in a certain
way, the application will close without any error messages.',
                'discovered_by'    => 'Sabrina',
                'person_type'      => PersonType::CUSTOMER->value,
                'occurred_on'      => '2004-04-14',
```

```
                'location'         => 'settings form',
                'initial_severity' => DefectSeverity::CRITICAL->value,
                'initial_priority' => DefectPriority::MOST_URGENT->dbValue(),
                'additional_info'  => 'please fix this soon',
                'analysis'         => [
                    'developer_description'   => 'It seems to be happening when you drag a p
arent item and drop it to its children.',
                    'classification_id'       => $classificationMap[IeeeFaultType::LOGIC->va
lue],
                    'date_originated'         => '2004-04-16',
                    'phases'                  => [OriginPhase::IMPLEMENTATION->value, Origin
Phase::TESTING->value],
                    'activity_originated'     => 'Testing',
                    'deliverable'             => 'the pop-up menu settings form',
                    'what_to_do'              => 'Prevent cyclic parent-to-child drag-and-dr
op in popup tree.',
                    'assigned_on'             => '2004-04-17',
                    'uniqueness'              => Uniqueness::CRITICAL->dbValue(),
                    'priority'                => DefectPriority::MOST_URGENT->dbValue(),
                    'severity'                => DefectSeverity::CRITICAL->value,
                    'workaround'              => 'Do not drag and drop.',
                    'proposed_fix'            => "Let's fix the pop-up menu module.",
                    'status'                  => DefectStatus::RESOLVED->value,
                    'fix_description'         => 'Fixed the pop-up menu function call.',
                    'fix_started_on'          => '2004-04-19',
                    'fix_completed_on'        => '2004-04-20',
                    'fix_impact'              => 'MMF 1.5 settings.cpp popup.cpp',
                    'fix_effort'              => 2.00,
                    'fix_cost'                => 60.00,
                    'resolution_description'  => 'The pop-up menu is fixed.',
                    'resolution_completed_on' => '2004-04-22',
                    'resolution_closed_on'    => '2004-04-23',
                    'cost_of_defect'          => 240.00,
                    'activity_found'          => 'sequential testing',
                ],
            ],
            [
                'product_id'       => $productMap['DefMan 2004 Core'],
                'environment'      => 'Linux 2.4, Apache 1.3, PHP 4.3, MySQL 4.0',
                'reporter_user_id' => $sabrinaId,
                'guest_name'       => null,
                'guest_email'      => null,
                'reported_at'      => '2004-04-18 09:15:00',
                'title'            => 'Precision Loss in EUR Effort Multiplier',
                'description'      => 'Fractional effort hours get truncated when stored in
smallint(6) fixeffort and fixcost columns.',
                'discovered_by'    => 'Julia',
                'person_type'      => PersonType::DEVELOPER->value,
                'occurred_on'      => '2004-04-18',
                'location'         => 'form_deffix.php',
                'initial_severity' => DefectSeverity::MAJOR->value,
                'initial_priority' => DefectPriority::MOST_URGENT->dbValue(),
                'additional_info'  => 'Affects IEEE cost reporting accuracy.',
                'analysis'         => [
                    'developer_description'   => 'Integer truncation in legacy MyISAM schema
 causes rounding fault.',
                    'classification_id'       => $classificationMap[IeeeFaultType::COMPUTATI
ONAL->value],
                    'date_originated'         => '2004-04-10',
                    'phases'                  => [OriginPhase::DESIGN->value],
                    'activity_originated'     => 'Database schema definition',
                    'deliverable'             => 'install/installdb.php',
                    'what_to_do'              => 'Migrate fixeffort and fixcost to DECIMAL(8
,2) / DECIMAL(10,2).',
                    'assigned_on'             => '2004-04-19',
```

```
                    'uniqueness'             => Uniqueness::NEW_DEFECT->dbValue(),
                    'priority'               => DefectPriority::MOST_URGENT->dbValue(),
                    'severity'               => DefectSeverity::MAJOR->value,
                    'workaround'             => 'Round effort to nearest whole hour.',
                    'proposed_fix'           => 'Alter table columns to DECIMAL in InnoDB.'
,
                    'status'                 => DefectStatus::OPENED->value,
                    'fix_description'        => 'Migrated schema to MySQL 8 InnoDB DECIMAL
columns.',
                    'fix_started_on'         => '2004-04-20',
                    'fix_completed_on'       => null,
                    'fix_impact'             => 'schema.sql, form_deffix.php',
                    'fix_effort'             => 6.50,
                    'fix_cost'               => 195.00,
                    'resolution_description'  => null,
                    'resolution_completed_on' => null,
                    'resolution_closed_on'    => null,
                    'cost_of_defect'         => 420.00,
                    'activity_found'         => 'schema review',
                ],
            ],
            [
                'product_id'       => $productMap['Katiska Auth Gateway 2.0'],
                'environment'      => 'Netscape 7.1 / Solaris X-Session',
                'reporter_user_id' => $sabrinaId,
                'guest_name'       => null,
                'guest_email'      => null,
                'reported_at'      => '2004-04-21 11:40:00',
                'title'            => 'Subroutine Mismatch in Session Logout Redirect',
                'description'      => 'Session state flag logged=1 remains active if user cl
oses browser tab without hitting logout.php.',
                'discovered_by'    => 'Sabrina',
                'person_type'      => PersonType::TESTER->value,
                'occurred_on'      => '2004-04-21',
                'location'         => 'login.php / logout.php',
                'initial_severity' => DefectSeverity::MINOR->value,
                'initial_priority' => DefectPriority::AVERAGE->dbValue(),
                'additional_info'  => 'User management shows stale Online badge.',
                'analysis'         => [
                    'developer_description'   => 'Missing session expiration heartbeat in au
thentication interface.',
                    'classification_id'       => $classificationMap[IeeeFaultType::INTERFACE
->value],
                    'date_originated'         => '2004-04-12',
                    'phases'                  => [OriginPhase::DESIGN->value, OriginPhase::I
MPLEMENTATION->value],
                    'activity_originated'     => 'Session handling implementation',
                    'deliverable'             => 'login.php, logout.php',
                    'what_to_do'              => 'Introduce expiring bearer sessions in user
_sessions table.',
                    'assigned_on'             => '2004-04-22',
                    'uniqueness'              => Uniqueness::NEW_DEFECT->dbValue(),
                    'priority'                => DefectPriority::AVERAGE->dbValue(),
                    'severity'                => DefectSeverity::MINOR->value,
                    'workaround'              => 'Manually log out via menubar.',
                    'proposed_fix'            => 'Use token expiration and session TTL.',
                    'status'                  => DefectStatus::FIXED->value,
                    'fix_description'         => 'Implemented token TTL and automatic online
 presence check.',
                    'fix_started_on'          => '2004-04-23',
                    'fix_completed_on'        => '2004-04-24',
                    'fix_impact'              => 'AuthController.php, user_sessions',
                    'fix_effort'              => 4.00,
                    'fix_cost'                => 120.00,
                    'resolution_description'  => 'Awaiting final QA sign-off.',
```

```
                    'resolution_completed_on' => null,
                    'resolution_closed_on'    => null,
                    'cost_of_defect'          => 180.00,
                    'activity_found'          => 'integration testing',
                ],
            ],
        ];

        $reportStmt = $this->pdo->prepare(
            'INSERT INTO `defect_reports`
             (`product_id`, `environment`, `reporter_user_id`, `guest_name`, `guest_email`,
`reported_at`,
              `title`, `description`, `discovered_by`, `person_type`, `occurred_on`, `locati
on`,
              `initial_severity`, `initial_priority`, `additional_info`)
             VALUES
             (:product_id, :environment, :reporter_user_id, :guest_name, :guest_email, :repo
rted_at,
              :title, :description, :discovered_by, :person_type, :occurred_on, :location,
              :initial_severity, :initial_priority, :additional_info)'
        );

        $analysisStmt = $this->pdo->prepare(
            'INSERT INTO `defect_analyses`
             (`report_id`, `developer_description`, `classification_id`, `date_originated`,
              `activity_originated`, `deliverable`, `what_to_do`, `assigned_on`, `uniqueness
`,
              `priority`, `severity`, `workaround`, `proposed_fix`, `status`, `fix_descripti
on`,
              `fix_started_on`, `fix_completed_on`, `fix_impact`, `fix_effort`, `fix_cost`,
              `resolution_description`, `resolution_completed_on`, `resolution_closed_on`,
              `cost_of_defect`, `activity_found`)
             VALUES
             (:report_id, :developer_description, :classification_id, :date_originated,
              :activity_originated, :deliverable, :what_to_do, :assigned_on, :uniqueness,
              :priority, :severity, :workaround, :proposed_fix, :status, :fix_description,
              :fix_started_on, :fix_completed_on, :fix_impact, :fix_effort, :fix_cost,
              :resolution_description, :resolution_completed_on, :resolution_closed_on,
              :cost_of_defect, :activity_found)'
        );

        $analysisPhaseStmt = $this->pdo->prepare(
            'INSERT IGNORE INTO `analysis_phases` (`report_id`, `phase_id`)
             VALUES (:report_id, :phase_id)'
        );

        foreach ($sampleDefects as $defect) {
            $analysis = $defect['analysis'];
            unset($defect['analysis']);

            $reportStmt->execute([
                ':product_id'       => $defect['product_id'],
                ':environment'      => $defect['environment'],
                ':reporter_user_id' => $defect['reporter_user_id'],
                ':guest_name'       => $defect['guest_name'],
                ':guest_email'      => $defect['guest_email'],
                ':reported_at'      => $defect['reported_at'],
                ':title'            => $defect['title'],
                ':description'      => $defect['description'],
                ':discovered_by'    => $defect['discovered_by'],
                ':person_type'      => $defect['person_type'],
                ':occurred_on'      => $defect['occurred_on'],
                ':location'         => $defect['location'],
                ':initial_severity' => $defect['initial_severity'],
                ':initial_priority' => $defect['initial_priority'],
```

```
                ':additional_info'  => $defect['additional_info'],
            ]);

            $reportId = (int) $this->pdo->lastInsertId();

            $analysisStmt->execute([
                ':report_id'               => $reportId,
                ':developer_description'   => $analysis['developer_description'],
                ':classification_id'       => $analysis['classification_id'],
                ':date_originated'         => $analysis['date_originated'],
                ':activity_originated'     => $analysis['activity_originated'],
                ':deliverable'             => $analysis['deliverable'],
                ':what_to_do'              => $analysis['what_to_do'],
                ':assigned_on'             => $analysis['assigned_on'],
                ':uniqueness'              => $analysis['uniqueness'],
                ':priority'                => $analysis['priority'],
                ':severity'                => $analysis['severity'],
                ':workaround'              => $analysis['workaround'],
                ':proposed_fix'            => $analysis['proposed_fix'],
                ':status'                  => $analysis['status'],
                ':fix_description'         => $analysis['fix_description'],
                ':fix_started_on'          => $analysis['fix_started_on'],
                ':fix_completed_on'        => $analysis['fix_completed_on'],
                ':fix_impact'              => $analysis['fix_impact'],
                ':fix_effort'              => $analysis['fix_effort'],
                ':fix_cost'                => $analysis['fix_cost'],
                ':resolution_description'  => $analysis['resolution_description'],
                ':resolution_completed_on' => $analysis['resolution_completed_on'],
                ':resolution_closed_on'    => $analysis['resolution_closed_on'],
                ':cost_of_defect'          => $analysis['cost_of_defect'],
                ':activity_found'          => $analysis['activity_found'],
            ]);

            foreach ($analysis['phases'] as $phaseName) {
                if (isset($phaseMap[$phaseName])) {
                    $analysisPhaseStmt->execute([
                        ':report_id' => $reportId,
                        ':phase_id'  => $phaseMap[$phaseName],
                    ]);
                }
            }
        }

        $this->pdo->commit();
    } catch (\Throwable $e) {
        if ($this->pdo->inTransaction()) {
            $this->pdo->rollBack();
        }
        throw $e;
    }
  }
}
```

```
================================================================================
SECTION 3.3 User, Company, Session & Password Reset Repository (backend/src/Repository/UserRepos
    itory.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\UserRepository.php
================================================================================
```

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;
```

```
use DefMan\Domain\Enum\UserLevel;
use PDO;

/**
 * User, Company, Session & Password Reset Repository (3NF Schema)
 * Refactored from: db.php, login.php, logout.php, form_apply.php, form_newuser.php,
 *                  form_lostpassword.php, form_userdetails.php, form_usermanagement.php
 */
final readonly class UserRepository
{
    public function __construct(private PDO $pdo) {}

    public function findById(int $id): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT
                u.`id`,
                u.`login`,
                u.`password`,
                u.`email`,
                u.`userlevel`,
                u.`company_id`,
                c.`name` AS `company`,
                u.`phone`,
                u.`name`,
                u.`activate`,
                CASE WHEN EXISTS (
                    SELECT 1 FROM `user_sessions` s
                    WHERE s.`user_id` = u.`id` AND s.`expires_at` > NOW()
                ) THEN 1 ELSE 0 END AS `logged`
             FROM `users` u
             LEFT JOIN `companies` c ON c.`id` = u.`company_id`
             WHERE u.`id` = :id
             LIMIT 1'
        );
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function findActiveByLogin(string $login): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT
                u.`id`,
                u.`login`,
                u.`password`,
                u.`email`,
                u.`userlevel`,
                u.`company_id`,
                c.`name` AS `company`,
                u.`phone`,
                u.`name`,
                u.`activate`,
                CASE WHEN EXISTS (
                    SELECT 1 FROM `user_sessions` s
                    WHERE s.`user_id` = u.`id` AND s.`expires_at` > NOW()
                ) THEN 1 ELSE 0 END AS `logged`
             FROM `users` u
             LEFT JOIN `companies` c ON c.`id` = u.`company_id`
             WHERE u.`login` = :login AND u.`activate` = 1
             LIMIT 1'
        );
        $stmt->execute([':login' => $login]);
        $row = $stmt->fetch();
```

```
        return $row ?: null;
    }

    public function findActiveByEmail(string $email): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT
                u.`id`,
                u.`login`,
                u.`password`,
                u.`email`,
                u.`userlevel`,
                u.`company_id`,
                c.`name` AS `company`,
                u.`phone`,
                u.`name`,
                u.`activate`,
                CASE WHEN EXISTS (
                    SELECT 1 FROM `user_sessions` s
                    WHERE s.`user_id` = u.`id` AND s.`expires_at` > NOW()
                ) THEN 1 ELSE 0 END AS `logged`
             FROM `users` u
             LEFT JOIN `companies` c ON c.`id` = u.`company_id`
             WHERE u.`email` = :email AND u.`activate` = 1 AND u.`userlevel` > 1
             LIMIT 1'
        );
        $stmt->execute([':email' => $email]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function existsByEmail(string $email, ?int $excludeUserId = null): bool
    {
        $sql = 'SELECT COUNT(*) FROM `users` WHERE `email` = :email';
        $params = [':email' => $email];
        if ($excludeUserId !== null) {
            $sql .= ' AND `id` != :exclude_id';
            $params[':exclude_id'] = $excludeUserId;
        }
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return (int) $stmt->fetchColumn() > 0;
    }

    public function existsByLogin(string $login): bool
    {
        $stmt = $this->pdo->prepare('SELECT COUNT(*) FROM `users` WHERE `login` = :login');
        $stmt->execute([':login' => $login]);
        return (int) $stmt->fetchColumn() > 0;
    }

    /**
     * Lists approved users (userlevel > 0) and pending applicants (userlevel = 0)
     * Online status (`logged`) is derived from active rows in `user_sessions`.
     */
    public function listAllByStatus(): array
    {
        $activeStmt = $this->pdo->query(
            'SELECT
                u.`id`,
                u.`login`,
                u.`email`,
                u.`userlevel`,
                u.`company_id`,
                c.`name` AS `company`,
```

```
            u.‘phone‘,
            u.‘name‘,
            u.‘activate‘,
            CASE WHEN EXISTS (
                SELECT 1 FROM ‘user_sessions‘ s
                WHERE s.‘user_id‘ = u.‘id‘ AND s.‘expires_at‘ > NOW()
            ) THEN 1 ELSE 0 END AS ‘logged‘
         FROM ‘users‘ u
         LEFT JOIN ‘companies‘ c ON c.‘id‘ = u.‘company_id‘
         WHERE u.‘userlevel‘ > 0
         ORDER BY u.‘userlevel‘ ASC, u.‘login‘ ASC’
    );
    $applyingStmt = $this->pdo->query(
        ’SELECT
            u.‘id‘,
            u.‘login‘,
            u.‘email‘,
            u.‘userlevel‘,
            u.‘company_id‘,
            c.‘name‘ AS ‘company‘,
            u.‘phone‘,
            u.‘name‘,
            u.‘activate‘,
            0 AS ‘logged‘
         FROM ‘users‘ u
         LEFT JOIN ‘companies‘ c ON c.‘id‘ = u.‘company_id‘
         WHERE u.‘userlevel‘ = 0
         ORDER BY u.‘id‘ ASC’
    );

    return [
        ’users’      => $activeStmt->fetchAll(),
        ’applicants’ => $applyingStmt->fetchAll(),
    ];
}

/**
 * Resolves a company name to ‘companies.id‘ (3NF lookup table), creating it if needed
 */
public function resolveCompanyId(?string $company): ?int
{
    $trimmed = trim((string) $company);
    if ($trimmed === ’’) {
        return null;
    }

    $stmt = $this->pdo->prepare(
        ’INSERT INTO ‘companies‘ (‘name‘) VALUES (:name)
         ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
    );
    $stmt->execute([’:name’ => $trimmed]);
    return (int) $this->pdo->lastInsertId();
}

/**
 * Creates a new user or applicant
 * Refactored from: form_apply.php (userlevel=0, activate=0) & form_newuser.php (activate=1)
 */
public function create(
    string $login,
    string $plainPassword,
    string $email,
    UserLevel $userLevel,
    ?string $company = null,
    ?string $phone = null,
```

```
        ?string $name = null,
        bool $activate = false
    ): int {
        $companyId = $this->resolveCompanyId($company);

        $stmt = $this->pdo->prepare(
            'INSERT INTO `users` (`login`, `password`, `email`, `userlevel`, `company_id`, `phon
    e`, `name`, `activate`)
             VALUES (:login, :password, :email, :userlevel, :company_id, :phone, :name, :activat
    e)'
        );
        $stmt->execute([
            ':login'      => $login,
            ':password'   => password_hash($plainPassword, PASSWORD_ARGON2ID),
            ':email'      => $email,
            ':userlevel'  => $userLevel->value,
            ':company_id' => $companyId,
            ':phone'      => $phone !== null && trim($phone) !== '' ? trim($phone) : null,
            ':name'       => $name !== null && trim($name) !== '' ? trim($name) : null,
            ':activate'   => $activate ? 1 : 0,
        ]);

        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Admin approval of pending user application (db.php?function=accept)
     */
    public function approveApplicant(int $id, UserLevel $assignedLevel = UserLevel::CUSTOMER): b
    ool
    {
        $stmt = $this->pdo->prepare(
            'UPDATE `users` SET `userlevel` = :level, `activate` = 1 WHERE `id` = :id'
        );
        $stmt->execute([
            ':level' => $assignedLevel->value,
            ':id'    => $id,
        ]);
        return $stmt->rowCount() > 0;
    }

    public function updateRole(int $id, UserLevel $level): bool
    {
        $stmt = $this->pdo->prepare('UPDATE `users` SET `userlevel` = :level WHERE `id` = :id');
        $stmt->execute([':level' => $level->value, ':id' => $id]);
        return $stmt->rowCount() > 0;
    }

    /**
     * Deletes a non-admin user or declines an applicant (db.php?function=delete)
     * Preserves reporter identity on `defect_reports` in `guest_name` and `guest_email`
     * prior to deletion so `chk_defect_reports_reporter_identity` is always satisfied.
     */
    public function delete(int $id): bool
    {
        $this->pdo->beginTransaction();
        try {
            $preserveStmt = $this->pdo->prepare(
                'UPDATE `defect_reports` dr
                 INNER JOIN `users` u ON u.`id` = dr.`reporter_user_id`
                 SET dr.`guest_name`       = COALESCE(dr.`guest_name`, u.`login`),
                     dr.`guest_email`      = COALESCE(dr.`guest_email`, u.`email`),
                     dr.`reporter_user_id` = NULL
                 WHERE dr.`reporter_user_id` = :id
                   AND u.`userlevel` != :admin'
```

```
            );
            $preserveStmt->execute([
                ':id'    => $id,
                ':admin' => UserLevel::ADMIN->value,
            ]);

            $stmt = $this->pdo->prepare('DELETE FROM `users` WHERE `id` = :id AND `userlevel` !=
 :admin');
            $stmt->execute([
                ':id'    => $id,
                ':admin' => UserLevel::ADMIN->value,
            ]);
            $deleted = $stmt->rowCount() > 0;
            $this->pdo->commit();
            return $deleted;
        } catch (\Throwable $e) {
            if ($this->pdo->inTransaction()) {
                $this->pdo->rollBack();
            }
            throw $e;
        }
    }

    public function updatePassword(int $userId, string $newPlainPassword): void
    {
        $stmt = $this->pdo->prepare('UPDATE `users` SET `password` = :password WHERE `id` = :id'
    );
        $stmt->execute([
            ':password' => password_hash($newPlainPassword, PASSWORD_ARGON2ID),
            ':id'       => $userId,
        ]);

        $clearResets = $this->pdo->prepare('DELETE FROM `password_resets` WHERE `user_id` = :use
    r_id');
        $clearResets->execute([':user_id' => $userId]);
    }

    public function updateEmail(int $userId, string $email): void
    {
        $stmt = $this->pdo->prepare('UPDATE `users` SET `email` = :email WHERE `id` = :id');
        $stmt->execute([':email' => $email, ':id' => $userId]);
    }

    public function updatePersonalDetails(int $userId, string $name, string $company, string $ph
    one): void
    {
        $companyId = $this->resolveCompanyId($company);

        $stmt = $this->pdo->prepare(
            'UPDATE `users` SET `name` = :name, `company_id` = :company_id, `phone` = :phone WHE
    RE `id` = :id'
        );
        $stmt->execute([
            ':name'       => $name !== '' ? $name : null,
            ':company_id' => $companyId,
            ':phone'      => $phone !== '' ? $phone : null,
            ':id'         => $userId,
        ]);
    }

    public function createSessionToken(int $userId, ?string $ipAddress, ?string $userAgent): str
    ing
    {
        $rawToken  = bin2hex(random_bytes(32));
        $tokenHash = hash('sha256', $rawToken);
```

```
        $expiresAt = (new \DateTimeImmutable('+12 hours'))->format('Y-m-d H:i:s');

        $stmt = $this->pdo->prepare(
            'INSERT INTO `user_sessions` (`user_id`, `token_hash`, `ip_address`, `user_agent`, `
    expires_at`)
             VALUES (:user_id, :token_hash, :ip_address, :user_agent, :expires_at)'
        );
        $stmt->execute([
            ':user_id'    => $userId,
            ':token_hash' => $tokenHash,
            ':ip_address' => $ipAddress,
            ':user_agent' => $userAgent ? substr($userAgent, 0, 255) : null,
            ':expires_at' => $expiresAt,
        ]);

        return $rawToken;
    }

    public function findUserBySessionToken(string $rawToken): ?array
    {
        $tokenHash = hash('sha256', $rawToken);
        $stmt = $this->pdo->prepare(
            'SELECT
                u.`id`,
                u.`login`,
                u.`email`,
                u.`userlevel`,
                u.`company_id`,
                c.`name` AS `company`,
                u.`phone`,
                u.`name`,
                u.`activate`,
                1 AS `logged`
             FROM `user_sessions` s
             INNER JOIN `users` u ON u.`id` = s.`user_id`
             LEFT JOIN `companies` c ON c.`id` = u.`company_id`
             WHERE s.`token_hash` = :token_hash
               AND s.`expires_at` > NOW()
               AND u.`activate` = 1
             LIMIT 1'
        );
        $stmt->execute([':token_hash' => $tokenHash]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function revokeSessionToken(string $rawToken, int $userId): void
    {
        $tokenHash = hash('sha256', $rawToken);
        $stmt = $this->pdo->prepare(
            'DELETE FROM `user_sessions` WHERE `token_hash` = :token_hash AND `user_id` = :user_
    id'
        );
        $stmt->execute([
            ':token_hash' => $tokenHash,
            ':user_id'    => $userId,
        ]);
    }

    public function createPasswordResetToken(int $userId): string
    {
        $rawToken  = bin2hex(random_bytes(32));
        $tokenHash = hash('sha256', $rawToken);
        $expiresAt = (new \DateTimeImmutable('+1 hour'))->format('Y-m-d H:i:s');
```

```
        $stmt = $this->pdo->prepare(
            'INSERT INTO `password_resets` (`user_id`, `token_hash`, `expires_at`)
             VALUES (:user_id, :token_hash, :expires_at)'
        );
        $stmt->execute([
            ':user_id'    => $userId,
            ':token_hash' => $tokenHash,
            ':expires_at' => $expiresAt,
        ]);

        return $rawToken;
    }
}
```

==========

SECTION 3.4 Product Catalog Repository (backend/src/Repository/ProductRepository.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\ProductRepository.php

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;

use PDO;
use RuntimeException;

/**
 * Product Repository (`products` table -- 3NF)
 * Refactored from: form_productdetails.php
 */
final readonly class ProductRepository
{
    public function __construct(private PDO $pdo) {}

    public function findAll(): array
    {
        $stmt = $this->pdo->query(
            'SELECT p.`id`, p.`name`, COUNT(dr.`id`) AS `report_count`
             FROM `products` p
             LEFT JOIN `defect_reports` dr ON dr.`product_id` = p.`id`
             GROUP BY p.`id`, p.`name`
             ORDER BY p.`name` ASC'
        );
        return $stmt->fetchAll();
    }

    public function findById(int $id): ?array
    {
        $stmt = $this->pdo->prepare('SELECT `id`, `name` FROM `products` WHERE `id` = :id LIMIT
    1');
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function findByName(string $name): ?array
    {
        $stmt = $this->pdo->prepare('SELECT `id`, `name` FROM `products` WHERE `name` = :name LI
    MIT 1');
        $stmt->execute([':name' => $name]);
        $row = $stmt->fetch();
        return $row ?: null;
```

```
    }

    public function create(string $name): int
    {
        $stmt = $this->pdo->prepare('INSERT INTO `products` (`name`) VALUES (:name)');
        $stmt->execute([':name' => trim(strip_tags($name))]);
        return (int) $this->pdo->lastInsertId();
    }

    public function delete(int $id): ?string
    {
        $product = $this->findById($id);
        if ($product === null) {
            return null;
        }

        $countStmt = $this->pdo->prepare('SELECT COUNT(*) FROM `defect_reports` WHERE `product_i
    d` = :id');
        $countStmt->execute([':id' => $id]);
        $reportCount = (int) $countStmt->fetchColumn();
        if ($reportCount > 0) {
            throw new RuntimeException(
                "Cannot delete product '{$product['name']}' because {$reportCount} defect report
    (s) reference it."
            );
        }

        $stmt = $this->pdo->prepare('DELETE FROM `products` WHERE `id` = :id');
        $stmt->execute([':id' => $id]);
        return (string) $product['name'];
    }
}
```

==========

SECTION 3.5 Product Controller (backend/src/Http/Controller/ProductController.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Controller\ProductControl
    ler.php

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Http\Controller;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Repository\ProductRepository;
use RuntimeException;

/**
 * Product Management Controller
 * Refactored from: form_productdetails.php
 */
final readonly class ProductController
{
    public function __construct(
        private ProductRepository $productRepository,
        private RbacMiddleware $rbac
    ) {}

    /**
     * GET /api/products (Available to all levels so Public/Customer forms can populate dropdown
    s)
     */
```

```
    public function listProducts(): array
    {
        return ['products' => $this->productRepository->findAll()];
    }

    /**
     * POST /api/admin/products (Refactored from form_productdetails.php?function=newproduct)
     */
    public function createProduct(array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);

        $name = trim(strip_tags((string) ($body['name'] ?? '')));
        if ($name === '') {
            http_response_code(422);
            return ['error' => 'Empty Product name!'];
        }

        if ($this->productRepository->findByName($name) !== null) {
            http_response_code(409);
            return ['error' => "Product '{$name}' already exists."];
        }

        $id = $this->productRepository->create($name);
        http_response_code(201);
        return [
            'id'      => $id,
            'name'    => $name,
            'message' => "You added new product: {$name} in database.",
        ];
    }

    /**
     * DELETE /api/admin/products/{id} (Refactored from form_productdetails.php?function=deletep
    roduct)
     */
    public function deleteProduct(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);

        try {
            $deletedName = $this->productRepository->delete($id);
        } catch (RuntimeException $e) {
            http_response_code(409);
            return ['error' => $e->getMessage()];
        }

        if ($deletedName === null) {
            http_response_code(404);
            return ['error' => 'Product not found.'];
        }

        return ['message' => "You deleted product: {$deletedName} from the database."];
    }
}


================================================================================
SECTION 3.6 Normalized Defect Report Repository (backend/src/Repository/DefectRepository.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\DefectRepository.ph
    p
================================================================================

<?php
declare(strict_types=1);
```

```
namespace DefMan\Repository;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\PersonType;
use PDO;

/**
 * Defect Report Repository (‘defect_reports‘ + 3NF joins)
 * Refactored from: form_raport.php, form_raportpub.php, form_raportlist.php,
 *                  form_list.php, form_defrap.php, db.php (deleteraport)
 */
final readonly class DefectRepository
{
    public function __construct(private PDO $pdo) {}

    /**
     * Resolves a product name to ‘products.id‘ (inserting if not yet cataloged)
     */
    private function resolveProductId(string $productName): int
    {
        $trimmed = trim(strip_tags($productName));
        if ($trimmed === ’’) {
            $trimmed = ’Unknown Product’;
        }

        $stmt = $this->pdo->prepare(
            ’INSERT INTO ‘products‘ (‘name‘) VALUES (:name)
             ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
        );
        $stmt->execute([’:name’ => $trimmed]);
        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Multi-column sorting & filtering by severity, priority, status, product, and customer.
     * Also computes ‘days_since_submitted‘ and deleted-user badge per form_raportlist.php.
     */
    public function findFiltered(
        ?string $severity = null,
        ?string $priority = null,
        ?string $status = null,
        ?string $product = null,
        ?int $customerId = null,
        string $sortBy = ’date’
    ): array {
        $conditions = [];
        $params     = [];

        if ($severity !== null && $severity !== ’’) {
            $conditions[]        = ’dr.‘initial_severity‘ = :severity’;
            $params[’:severity’] = $severity;
        }

        if ($priority !== null && $priority !== ’’) {
            $normalizedPriority  = DefectPriority::fromInput($priority)->dbValue();
            $conditions[]        = ’dr.‘initial_priority‘ = :priority’;
            $params[’:priority’] = $normalizedPriority;
        }

        if ($status !== null && $status !== ’’) {
            if ($status === ’unassigned’) {
                $conditions[] = ’da.‘status‘ IS NULL’;
            } else {
```

```
            $conditions[]      = 'da.`status` = :status';
            $params[':status'] = $status;
        }
    }

    if ($product !== null && $product !== '') {
        $conditions[]       = 'p.`name` = :product';
        $params[':product'] = $product;
    }

    if ($customerId !== null) {
        $conditions[]           = 'dr.`reporter_user_id` = :customer_id';
        $params[':customer_id'] = $customerId;
    }

    $whereSql = count($conditions) > 0 ? 'WHERE ' . implode(' AND ', $conditions) : '';

    $orderBySql = match ($sortBy) {
        'product'  => 'p.`name` ASC, dr.`id` DESC',
        'contact'  => "COALESCE(u.`login`, dr.`guest_name`, '') DESC, dr.`id` DESC",
        'severity' => "FIELD(dr.`initial_severity`, 'critical', 'major', 'minor') ASC, dr.`r
eported_at` DESC",
        'priority', 'all' => "FIELD(dr.`initial_priority`, 'most_urgent', 'average', 'less_u
rgent') ASC, dr.`reported_at` DESC",
        default    => 'dr.`reported_at` DESC, dr.`id` DESC',
    };

    $sql = "
        SELECT
            dr.`id` AS `ID`,
            p.`name` AS `Product`,
            dr.`product_id`,
            dr.`environment` AS `Environment`,
            dr.`reporter_user_id` AS `CustomerID`,
            COALESCE(u.`login`, dr.`guest_name`, '') AS `Who`,
            COALESCE(u.`email`, dr.`guest_email`, '') AS `Contactemail`,
            DATE_FORMAT(dr.`reported_at`, '%Y-%m-%d %H:%i') AS `Whenreported`,
            dr.`title` AS `Title`,
            dr.`description` AS `Description`,
            dr.`discovered_by` AS `Discoveredby`,
            dr.`person_type` AS `Persontype`,
            dr.`occurred_on` AS `Occured`,
            dr.`location` AS `Location`,
            dr.`initial_severity` AS `Severity`,
            CASE dr.`initial_priority`
                WHEN 'most_urgent' THEN 'm_urgent'
                WHEN 'less_urgent' THEN 'l_urgent'
                ELSE 'average'
            END AS `Priority`,
            CASE dr.`initial_priority`
                WHEN 'most_urgent' THEN 'most urgent'
                WHEN 'less_urgent' THEN 'less urgent'
                ELSE 'average'
            END AS `PriorityLabel`,
            DATE(dr.`reported_at`) AS `Submitted`,
            DATEDIFF(CURDATE(), DATE(dr.`reported_at`)) AS `days_since_submitted`,
            dr.`additional_info` AS `Addinfo`,
            CASE
                WHEN dr.`reporter_user_id` IS NULL AND dr.`person_type` = 'Customer' AND dr.
`guest_name` IS NOT NULL
                     AND NOT EXISTS (SELECT 1 FROM `users` u2 WHERE u2.`login` = dr.`guest_n
ame`)
                THEN 0
                WHEN dr.`reporter_user_id` IS NOT NULL AND u.`id` IS NULL
                THEN 1
```

```
                ELSE 0
            END AS ‘is_deleted_user‘,
            fc.‘code‘ AS ‘classification‘,
            COALESCE(da.‘status‘, ’submitted’) AS ‘analysis_status‘,
            da.‘severity‘ AS ‘developer_severity‘,
            CASE da.‘priority‘
                WHEN ’most_urgent’ THEN ’most urgent’
                WHEN ’less_urgent’ THEN ’less urgent’
                WHEN ’average’     THEN ’average’
                ELSE NULL
            END AS ‘developer_priority‘,
            da.‘fix_effort‘ AS ‘fixeffort‘,
            da.‘fix_cost‘ AS ‘fixcost‘,
            da.‘cost_of_defect‘ AS ‘costofdefect‘,
            da.‘assigned_on‘ AS ‘assigned‘,
            da.‘fix_completed_on‘ AS ‘fixcomplete‘,
            da.‘resolution_completed_on‘ AS ‘resocomplete‘,
            da.‘resolution_closed_on‘ AS ‘resoclosed‘
        FROM ‘defect_reports‘ dr
        INNER JOIN ‘products‘ p ON p.‘id‘ = dr.‘product_id‘
        LEFT JOIN ‘users‘ u ON u.‘id‘ = dr.‘reporter_user_id‘
        LEFT JOIN ‘defect_analyses‘ da ON da.‘report_id‘ = dr.‘id‘
        LEFT JOIN ‘fault_classifications‘ fc ON fc.‘id‘ = da.‘classification_id‘
        {$whereSql}
        ORDER BY {$orderBySql}
    ";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll();
}

public function findById(int $id): ?array
{
    $stmt = $this->pdo->prepare(
        "SELECT
            dr.‘id‘ AS ‘ID‘,
            p.‘name‘ AS ‘Product‘,
            dr.‘product_id‘,
            dr.‘environment‘ AS ‘Environment‘,
            dr.‘reporter_user_id‘ AS ‘CustomerID‘,
            COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘Who‘,
            COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘Contactemail‘,
            DATE_FORMAT(dr.‘reported_at‘, ’%Y-%m-%d %H:%i’) AS ‘Whenreported‘,
            dr.‘title‘ AS ‘Title‘,
            dr.‘description‘ AS ‘Description‘,
            dr.‘discovered_by‘ AS ‘Discoveredby‘,
            dr.‘person_type‘ AS ‘Persontype‘,
            dr.‘occurred_on‘ AS ‘Occured‘,
            dr.‘location‘ AS ‘Location‘,
            dr.‘initial_severity‘ AS ‘Severity‘,
            CASE dr.‘initial_priority‘
                WHEN ’most_urgent’ THEN ’m_urgent’
                WHEN ’less_urgent’ THEN ’l_urgent’
                ELSE ’average’
            END AS ‘Priority‘,
            CASE dr.‘initial_priority‘
                WHEN ’most_urgent’ THEN ’most urgent’
                WHEN ’less_urgent’ THEN ’less urgent’
                ELSE ’average’
            END AS ‘PriorityLabel‘,
            DATE(dr.‘reported_at‘) AS ‘Submitted‘,
            DATEDIFF(CURDATE(), DATE(dr.‘reported_at‘)) AS ‘days_since_submitted‘,
            dr.‘additional_info‘ AS ‘Addinfo‘
         FROM ‘defect_reports‘ dr
```

```
            INNER JOIN ‘products‘ p ON p.‘id‘ = dr.‘product_id‘
            LEFT JOIN ‘users‘ u ON u.‘id‘ = dr.‘reporter_user_id‘
            WHERE dr.‘id‘ = :id
            LIMIT 1"
        );
        $stmt->execute([’:id’ => $id]);
        $report = $stmt->fetch();
        if (!$report) {
            return null;
        }

        $analysisStmt = $this->pdo->prepare(
            "SELECT
                da.‘report_id‘ AS ‘id‘,
                da.‘developer_description‘ AS ‘devdesc‘,
                fc.‘code‘ AS ‘classification‘,
                da.‘date_originated‘ AS ‘dateoriginated‘,
                ph.‘phases_csv‘ AS ‘phase‘,
                da.‘activity_originated‘ AS ‘actoriginated‘,
                da.‘deliverable‘,
                da.‘what_to_do‘ AS ‘whattodo‘,
                DATE(dr.‘reported_at‘) AS ‘submitted‘,
                da.‘assigned_on‘ AS ‘assigned‘,
                CASE da.‘uniqueness‘
                    WHEN ’duplicate’ THEN ’Already submitted’
                    ELSE ’New defect’
                END AS ‘uniqueness‘,
                da.‘duplicate_of_report_id‘,
                CASE da.‘priority‘
                    WHEN ’most_urgent’ THEN ’most urgent’
                    WHEN ’less_urgent’ THEN ’less urgent’
                    ELSE ’average’
                END AS ‘priority‘,
                da.‘severity‘,
                da.‘workaround‘,
                da.‘proposed_fix‘ AS ‘proposedfix‘,
                da.‘status‘,
                da.‘fix_description‘ AS ‘fixdesc‘,
                da.‘fix_started_on‘ AS ‘fixstarted‘,
                da.‘fix_completed_on‘ AS ‘fixcomplete‘,
                da.‘fix_impact‘ AS ‘fiximpact‘,
                da.‘fix_effort‘ AS ‘fixeffort‘,
                da.‘fix_cost‘ AS ‘fixcost‘,
                da.‘resolution_description‘ AS ‘resodesc‘,
                da.‘resolution_completed_on‘ AS ‘resocomplete‘,
                da.‘resolution_closed_on‘ AS ‘resoclosed‘,
                da.‘cost_of_defect‘ AS ‘costofdefect‘,
                da.‘activity_found‘ AS ‘actfound‘
            FROM ‘defect_analyses‘ da
            INNER JOIN ‘defect_reports‘ dr ON dr.‘id‘ = da.‘report_id‘
            LEFT JOIN ‘fault_classifications‘ fc ON fc.‘id‘ = da.‘classification_id‘
            LEFT JOIN (
                SELECT
                    ap.‘report_id‘,
                    GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
                FROM ‘analysis_phases‘ ap
                INNER JOIN ‘sdlc_phases‘ sp ON sp.‘id‘ = ap.‘phase_id‘
                WHERE ap.‘report_id‘ = :report_id
                GROUP BY ap.‘report_id‘
            ) ph ON ph.‘report_id‘ = da.‘report_id‘
            WHERE da.‘report_id‘ = :id
            LIMIT 1"
        );
        $analysisStmt->execute([’:id’ => $id, ’:report_id’ => $id]);
        $analysis = $analysisStmt->fetch();
```

```
        if ($analysis) {
            $analysis['phases'] = !empty($analysis['phase'])
                ? explode(',', (string) $analysis['phase'])
                : [];
            $report['analysis'] = $analysis;
        } else {
            $report['analysis'] = null;
        }

        return $report;
    }

    /**
     * Creates a new normalized defect report (`defect_reports`)
     */
    public function create(array $data): int
    {
        $severity   = DefectSeverity::from($data['severity'] ?? 'minor');
        $priority   = DefectPriority::fromInput($data['priority'] ?? 'average');
        $personType = PersonType::from($data['persontype'] ?? 'Customer');
        $productId  = $this->resolveProductId((string) $data['product']);

        $reporterUserId = isset($data['customer_id']) && $data['customer_id'] !== null
            ? (int) $data['customer_id']
            : null;

        $guestName  = $reporterUserId === null
            ? substr(trim(strip_tags((string) ($data['who'] ?? 'Anonymous Guest'))), 0, 100)
            : null;
        $guestEmail = $reporterUserId === null
            ? trim(strip_tags((string) ($data['contact_email'] ?? '')))
            : null;

        $stmt = $this->pdo->prepare(
            'INSERT INTO `defect_reports`
             (`product_id`, `environment`, `reporter_user_id`, `guest_name`, `guest_email`, `rep
    orted_at`,
              `title`, `description`, `discovered_by`, `person_type`, `occurred_on`, `location`,
              `initial_severity`, `initial_priority`, `additional_info`)
             VALUES
             (:product_id, :environment, :reporter_user_id, :guest_name, :guest_email, :reported
    _at,
              :title, :description, :discovered_by, :person_type, :occurred_on, :location,
              :initial_severity, :initial_priority, :additional_info)'
        );

        $stmt->execute([
            ':product_id'       => $productId,
            ':environment'      => trim(strip_tags((string) ($data['environment'] ?? ''))),
            ':reporter_user_id' => $reporterUserId,
            ':guest_name'       => $guestName,
            ':guest_email'      => $guestEmail,
            ':reported_at'      => date('Y-m-d H:i:s'),
            ':title'            => trim(strip_tags((string) $data['title'])),
            ':description'      => trim(strip_tags((string) $data['description'])),
            ':discovered_by'    => trim(strip_tags((string) ($data['discovered_by'] ?? ''))),
            ':person_type'      => $personType->value,
            ':occurred_on'      => $data['occured'],
            ':location'         => trim(strip_tags((string) ($data['location'] ?? ''))),
            ':initial_severity' => $severity->value,
            ':initial_priority' => $priority->dbValue(),
            ':additional_info'  => trim(strip_tags((string) ($data['addinfo'] ?? ''))),
        ]);

        return (int) $this->pdo->lastInsertId();
```

```
    }

    /**
     * Updates an existing defect report (‘defect_reports‘)
     */
    public function update(int $id, array $data): bool
    {
        $severity   = DefectSeverity::from($data[’severity’]);
        $priority   = DefectPriority::fromInput($data[’priority’]);
        $personType = PersonType::from($data[’persontype’]);
        $productId  = $this->resolveProductId((string) $data[’product’]);

        $stmt = $this->pdo->prepare(
            ’UPDATE ‘defect_reports‘ SET
                ‘product_id‘       = :product_id,
                ‘environment‘      = :environment,
                ‘title‘            = :title,
                ‘description‘      = :description,
                ‘discovered_by‘    = :discovered_by,
                ‘person_type‘      = :person_type,
                ‘occurred_on‘      = :occurred_on,
                ‘location‘         = :location,
                ‘initial_severity‘ = :initial_severity,
                ‘initial_priority‘ = :initial_priority,
                ‘additional_info‘  = :additional_info
             WHERE ‘id‘ = :id’
        );

        $stmt->execute([
            ’:product_id’       => $productId,
            ’:environment’      => trim(strip_tags((string) ($data[’environment’] ?? ’’))),
            ’:title’            => trim(strip_tags((string) $data[’title’])),
            ’:description’      => trim(strip_tags((string) $data[’description’])),
            ’:discovered_by’    => trim(strip_tags((string) ($data[’discovered_by’] ?? ’’))),
            ’:person_type’      => $personType->value,
            ’:occurred_on’      => $data[’occured’],
            ’:location’         => trim(strip_tags((string) ($data[’location’] ?? ’’))),
            ’:initial_severity’ => $severity->value,
            ’:initial_priority’ => $priority->dbValue(),
            ’:additional_info’  => trim(strip_tags((string) ($data[’addinfo’] ?? ’’))),
            ’:id’               => $id,
        ]);

        return $stmt->rowCount() >= 0;
    }

    /**
     * Deletes a defect report and cascades to ‘defect_analyses‘ & ‘analysis_phases‘
     */
    public function delete(int $id): bool
    {
        $stmt = $this->pdo->prepare(’DELETE FROM ‘defect_reports‘ WHERE ‘id‘ = :id’);
        $stmt->execute([’:id’ => $id]);
        return $stmt->rowCount() > 0;
    }
}
```

==========

SECTION 3.7 Normalized Defect Analysis & SDLC Phase Repository (backend/src/Repository/AnalysisR
epository.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\AnalysisRepository.
php
==========

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\DefectStatus;
use DefMan\Domain\Enum\IeeeFaultType;
use DefMan\Domain\Enum\Uniqueness;
use PDO;

/**
 * Defect Analysis, Fix & Resolution Repository (‘defect_analyses‘, ‘fault_classifications‘, ‘sd
    lc_phases‘, ‘analysis_phases‘ -- 3NF)
 * Refactored from: form_alist.php, form_analyze.php, form_analyzed.php,
 *                  form_deffix.php, form_defreso.php, db.php (deleteanalyze)
 */
final readonly class AnalysisRepository
{
    public function __construct(private PDO $pdo) {}

    /**
     * Resolves an IEEE 1044-1992 fault classification code to ‘fault_classifications.id‘
     */
    private function resolveClassificationId(string $code): int
    {
        $trimmed = trim(strip_tags($code));
        if ($trimmed === ’’) {
            $trimmed = IeeeFaultType::OTHER->value;
        }

        $stmt = $this->pdo->prepare(
            ’INSERT INTO ‘fault_classifications‘ (‘code‘) VALUES (:code)
             ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
        );
        $stmt->execute([’:code’ => $trimmed]);
        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Syncs the 1NF ‘analysis_phases‘ junction table for a given ‘report_id‘
     *
     * @param list<string> $phases
     */
    private function syncAnalysisPhases(int $reportId, array $phases): void
    {
        $deleteStmt = $this->pdo->prepare(’DELETE FROM ‘analysis_phases‘ WHERE ‘report_id‘ = :re
    port_id’);
        $deleteStmt->execute([’:report_id’ => $reportId]);

        $phaseUpsertStmt = $this->pdo->prepare(
            ’INSERT INTO ‘sdlc_phases‘ (‘name‘) VALUES (:name)
             ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
        );
        $junctionStmt = $this->pdo->prepare(
            ’INSERT IGNORE INTO ‘analysis_phases‘ (‘report_id‘, ‘phase_id‘) VALUES (:report_id,
    :phase_id)’
        );

        foreach ($phases as $rawPhase) {
            $phaseName = trim(strip_tags((string) $rawPhase));
            if ($phaseName === ’’) {
                continue;
            }
```

```
            $phaseUpsertStmt->execute([':name' => $phaseName]);
            $phaseId = (int) $this->pdo->lastInsertId();
            $junctionStmt->execute([
                ':report_id' => $reportId,
                ':phase_id'  => $phaseId,
            ]);
        }
    }

    /**
     * Returns the full Analyzer Queue with lead-time metrics:
     * - submitted_to_assigned_days
     * - submitted_to_fixed_days
     * - submitted_to_resolved_days
     * - assigned_to_fixed_days
     * - assigned_to_resolved_days
     */
    public function getAnalyzerQueue(?string $sort = 'all'): array
    {
        $whereSql = '';
        $params   = [];

        if (in_array($sort, ['critical', 'major', 'minor'], true)) {
            $whereSql        = 'WHERE COALESCE(da.`severity`, dr.`initial_severity`) = :sort';
            $params[':sort'] = $sort;
        } elseif (in_array($sort, ['most urgent', 'average', 'less urgent', 'most_urgent', 'less
    _urgent'], true)) {
            $whereSql        = 'WHERE COALESCE(da.`priority`, dr.`initial_priority`) = :sort';
            $params[':sort'] = DefectPriority::fromInput($sort)->dbValue();
        } elseif (in_array($sort, ['submitted', 'assigned', 'postponed', 'opened', 'fixed', 'res
    olved', 'closed'], true)) {
            $whereSql        = "WHERE COALESCE(da.`status`, 'submitted') = :sort";
            $params[':sort'] = $sort;
        }

        $orderBySql = match ($sort) {
            'date'  => 'dr.`reported_at` DESC, dr.`id` DESC',
            default => "FIELD(COALESCE(da.`priority`, dr.`initial_priority`, 'average'), 'most_u
    rgent', 'average', 'less_urgent') ASC, dr.`reported_at` DESC",
        };

        $sql = "
            SELECT
                dr.`id` AS `id`,
                p.`name` AS `product`,
                dr.`title` AS `title`,
                dr.`description` AS `report_description`,
                dr.`environment` AS `environment`,
                dr.`occurred_on` AS `occured`,
                COALESCE(u.`email`, dr.`guest_email`, '') AS `contact_email`,
                COALESCE(u.`login`, dr.`guest_name`, '') AS `who`,
                dr.`initial_severity` AS `report_severity`,
                CASE dr.`initial_priority`
                    WHEN 'most_urgent' THEN 'm_urgent'
                    WHEN 'less_urgent' THEN 'l_urgent'
                    ELSE 'average'
                END AS `report_priority`,
                DATE(dr.`reported_at`) AS `report_submitted`,
                da.`developer_description` AS `devdesc`,
                COALESCE(fc.`code`, 'Other') AS `classification`,
                da.`date_originated` AS `dateoriginated`,
                ph.`phases_csv` AS `phase`,
                da.`activity_originated` AS `actoriginated`,
                da.`deliverable`,
                da.`what_to_do` AS `whattodo`,
```

```
            DATE(dr.‘reported_at‘) AS ‘submitted‘,
            da.‘assigned_on‘ AS ‘assigned‘,
            CASE da.‘uniqueness‘
                WHEN ’duplicate’ THEN ’Already submitted’
                ELSE ’New defect’
            END AS ‘uniqueness‘,
            da.‘duplicate_of_report_id‘,
            COALESCE(da.‘severity‘, dr.‘initial_severity‘) AS ‘severity‘,
            CASE COALESCE(da.‘priority‘, dr.‘initial_priority‘)
                WHEN ’most_urgent’ THEN ’most urgent’
                WHEN ’less_urgent’ THEN ’less urgent’
                ELSE ’average’
            END AS ‘priority‘,
            da.‘workaround‘,
            da.‘proposed_fix‘ AS ‘proposedfix‘,
            COALESCE(da.‘status‘, ’submitted’) AS ‘status‘,
            da.‘fix_description‘ AS ‘fixdesc‘,
            da.‘fix_started_on‘ AS ‘fixstarted‘,
            da.‘fix_completed_on‘ AS ‘fixcomplete‘,
            da.‘fix_impact‘ AS ‘fiximpact‘,
            COALESCE(da.‘fix_effort‘, 0.00) AS ‘fixeffort‘,
            COALESCE(da.‘fix_cost‘, 0.00) AS ‘fixcost‘,
            da.‘resolution_description‘ AS ‘resodesc‘,
            da.‘resolution_completed_on‘ AS ‘resocomplete‘,
            da.‘resolution_closed_on‘ AS ‘resoclosed‘,
            COALESCE(da.‘cost_of_defect‘, 0.00) AS ‘costofdefect‘,
            da.‘activity_found‘ AS ‘actfound‘,
            DATEDIFF(CURDATE(), DATE(dr.‘reported_at‘)) AS ‘days_open‘,
            CASE WHEN da.‘assigned_on‘ IS NOT NULL
                 THEN DATEDIFF(da.‘assigned_on‘, DATE(dr.‘reported_at‘))
                 ELSE NULL END AS ‘submitted_to_assigned_days‘,
            CASE WHEN da.‘fix_completed_on‘ IS NOT NULL
                 THEN DATEDIFF(da.‘fix_completed_on‘, DATE(dr.‘reported_at‘))
                 ELSE NULL END AS ‘submitted_to_fixed_days‘,
            CASE WHEN da.‘resolution_completed_on‘ IS NOT NULL
                 THEN DATEDIFF(da.‘resolution_completed_on‘, DATE(dr.‘reported_at‘))
                 ELSE NULL END AS ‘submitted_to_resolved_days‘,
            CASE WHEN da.‘fix_completed_on‘ IS NOT NULL AND da.‘assigned_on‘ IS NOT NULL
                 THEN DATEDIFF(da.‘fix_completed_on‘, da.‘assigned_on‘)
                 ELSE NULL END AS ‘assigned_to_fixed_days‘,
            CASE WHEN da.‘resolution_completed_on‘ IS NOT NULL AND da.‘assigned_on‘ IS NOT N
ULL
                 THEN DATEDIFF(da.‘resolution_completed_on‘, da.‘assigned_on‘)
                 ELSE NULL END AS ‘assigned_to_resolved_days‘
        FROM ‘defect_reports‘ dr
        INNER JOIN ‘products‘ p ON p.‘id‘ = dr.‘product_id‘
        LEFT JOIN ‘users‘ u ON u.‘id‘ = dr.‘reporter_user_id‘
        LEFT JOIN ‘defect_analyses‘ da ON da.‘report_id‘ = dr.‘id‘
        LEFT JOIN ‘fault_classifications‘ fc ON fc.‘id‘ = da.‘classification_id‘
        LEFT JOIN (
            SELECT
                ap.‘report_id‘,
                GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
            FROM ‘analysis_phases‘ ap
            INNER JOIN ‘sdlc_phases‘ sp ON sp.‘id‘ = ap.‘phase_id‘
            GROUP BY ap.‘report_id‘
        ) ph ON ph.‘report_id‘ = dr.‘id‘
        {$whereSql}
        ORDER BY {$orderBySql}
    ";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute($params);
    $rows = $stmt->fetchAll();
```

```
        foreach ($rows as &$row) {
            $faultEnum = IeeeFaultType::fromLegacyString((string) $row['classification']);
            $row['classification_color'] = sprintf('#%06X', $faultEnum->hexColor());
            $row['phases'] = !empty($row['phase']) ? explode(',', (string) $row['phase']) : [];
        }

        return $rows;
    }

    /**
     * Upserts initial developer analysis & IEEE (1992) classification (‘defect_analyses‘ + ‘ana
    lysis_phases‘)
     */
    public function upsertAnalysis(int $defectId, array $data): void
    {
        $phaseList = [];
        if (is_array($data['phases'] ?? null)) {
            $phaseList = array_values(array_filter(array_map('strval', $data['phases'])));
        } elseif (!empty($data['phase'])) {
            $phaseList = array_values(array_filter(array_map('trim', explode(',', (string) $data
    ['phase']))));
        }

        $classificationCode = !empty($data['classification_other'])
            ? trim(strip_tags((string) $data['classification_other']))
            : IeeeFaultType::fromLegacyString((string) ($data['classification'] ?? 'Logic'))->va
    lue;

        $uniquenessEnum = Uniqueness::fromInput((string) ($data['uniqueness'] ?? 'New defect'));
        $uniquenessDb   = $uniquenessEnum->dbValue();

        $severity = $uniquenessEnum === Uniqueness::CRITICAL
            ? DefectSeverity::CRITICAL->value
            : DefectSeverity::from($data['severity'] ?? 'minor')->value;

        $priorityDb = DefectPriority::fromInput((string) ($data['priority'] ?? 'average'))->dbVa
    lue();
        $status     = DefectStatus::from($data['status'] ?? 'opened')->value;

        $duplicateOfReportId = !empty($data['duplicate_of_report_id'])
            ? (int) $data['duplicate_of_report_id']
            : null;

        $this->pdo->beginTransaction();
        try {
            $classificationId = $this->resolveClassificationId($classificationCode);

            $stmt = $this->pdo->prepare(
                'INSERT INTO ‘defect_analyses‘
                 (‘report_id‘, ‘developer_description‘, ‘classification_id‘, ‘date_originated‘,
                  ‘activity_originated‘, ‘deliverable‘, ‘what_to_do‘, ‘assigned_on‘, ‘uniqueness
    ‘,
                  ‘duplicate_of_report_id‘, ‘priority‘, ‘severity‘, ‘workaround‘, ‘proposed_fix‘
    ,
                  ‘status‘, ‘activity_found‘)
                 VALUES
                 (:report_id, :developer_description, :classification_id, :date_originated,
                  :activity_originated, :deliverable, :what_to_do, :assigned_on, :uniqueness,
                  :duplicate_of_report_id, :priority, :severity, :workaround, :proposed_fix,
                  :status, :activity_found)
                 ON DUPLICATE KEY UPDATE
                  ‘developer_description‘  = VALUES(‘developer_description‘),
                  ‘classification_id‘      = VALUES(‘classification_id‘),
                  ‘date_originated‘        = VALUES(‘date_originated‘),
                  ‘activity_originated‘    = VALUES(‘activity_originated‘),
```

```
                `deliverable`            = VALUES(`deliverable`),
                `what_to_do`             = VALUES(`what_to_do`),
                `assigned_on`            = VALUES(`assigned_on`),
                `uniqueness`             = VALUES(`uniqueness`),
                `duplicate_of_report_id` = VALUES(`duplicate_of_report_id`),
                `priority`               = VALUES(`priority`),
                `severity`               = VALUES(`severity`),
                `workaround`             = VALUES(`workaround`),
                `proposed_fix`           = VALUES(`proposed_fix`),
                `status`                 = VALUES(`status`),
                `activity_found`         = VALUES(`activity_found`)'
        );

        $stmt->execute([
            ':report_id'              => $defectId,
            ':developer_description'  => trim(strip_tags((string) $data['devdesc'])),
            ':classification_id'      => $classificationId,
            ':date_originated'        => $data['dateoriginated'],
            ':activity_originated'    => trim(strip_tags((string) $data['actoriginated'])),
            ':deliverable'            => trim(strip_tags((string) $data['deliverable'])),
            ':what_to_do'             => trim(strip_tags((string) ($data['whattodo'] ?? ''))
),
            ':assigned_on'            => $data['assigned'],
            ':uniqueness'             => $uniquenessDb,
            ':duplicate_of_report_id' => $duplicateOfReportId,
            ':priority'               => $priorityDb,
            ':severity'               => $severity,
            ':workaround'             => trim(strip_tags((string) ($data['workaround'] ?? ''
))),
            ':proposed_fix'           => trim(strip_tags((string) $data['proposedfix'])),
            ':status'                 => $status,
            ':activity_found'         => trim(strip_tags((string) $data['actfound'])),
        ]);

        $this->syncAnalysisPhases($defectId, $phaseList);
        $this->pdo->commit();
    } catch (\Throwable $e) {
        if ($this->pdo->inTransaction()) {
            $this->pdo->rollBack();
        }
        throw $e;
    }
}

/**
 * Upserts defect fix effort, monetary cost & resolution details (`defect_analyses`)
 * Refactored from: form_deffix.php & form_defreso.php
 */
public function upsertResolutionAndCost(int $defectId, array $data): void
{
    $status = isset($data['status'])
        ? DefectStatus::from((string) $data['status'])->value
        : DefectStatus::RESOLVED->value;

    $stmt = $this->pdo->prepare(
        'INSERT INTO `defect_analyses`
         (`report_id`, `status`, `fix_description`, `fix_started_on`, `fix_completed_on`, `f
ix_impact`,
          `fix_effort`, `fix_cost`, `resolution_description`, `resolution_completed_on`, `re
solution_closed_on`, `cost_of_defect`)
         VALUES
         (:report_id, :status, :fix_description, :fix_started_on, :fix_completed_on, :fix_im
pact,
          :fix_effort, :fix_cost, :resolution_description, :resolution_completed_on, :resolu
tion_closed_on, :cost_of_defect)
```

```
            ON DUPLICATE KEY UPDATE
             `status`                = VALUES(`status`),
             `fix_description`       = COALESCE(VALUES(`fix_description`), `fix_description`)
    ,
             `fix_started_on`        = COALESCE(VALUES(`fix_started_on`), `fix_started_on`),
             `fix_completed_on`      = COALESCE(VALUES(`fix_completed_on`), `fix_completed_on
    `),
             `fix_impact`            = COALESCE(VALUES(`fix_impact`), `fix_impact`),
             `fix_effort`            = COALESCE(VALUES(`fix_effort`), `fix_effort`),
             `fix_cost`              = COALESCE(VALUES(`fix_cost`), `fix_cost`),
             `resolution_description`  = COALESCE(VALUES(`resolution_description`), `resolution
    _description`),
             `resolution_completed_on` = COALESCE(VALUES(`resolution_completed_on`), `resolutio
    n_completed_on`),
             `resolution_closed_on`    = COALESCE(VALUES(`resolution_closed_on`), `resolution_c
    losed_on`),
             `cost_of_defect`        = COALESCE(VALUES(`cost_of_defect`), `cost_of_defect`)'
        );

        $stmt->execute([
            ':report_id'              => $defectId,
            ':status'                 => $status,
            ':fix_description'        => isset($data['fixdesc']) ? trim(strip_tags((string) $da
    ta['fixdesc'])) : null,
            ':fix_started_on'         => !empty($data['fixstarted']) ? $data['fixstarted'] : nu
    ll,
            ':fix_completed_on'       => !empty($data['fixcomplete']) ? $data['fixcomplete'] :
    null,
            ':fix_impact'             => isset($data['fiximpact']) ? trim(strip_tags((string) $
    data['fiximpact'])) : null,
            ':fix_effort'             => isset($data['fixeffort']) ? (float) $data['fixeffort']
     : null,
            ':fix_cost'               => isset($data['fixcost']) ? (float) $data['fixcost'] : n
    ull,
            ':resolution_description'  => isset($data['resodesc']) ? trim(strip_tags((string) $d
    ata['resodesc'])) : null,
            ':resolution_completed_on' => !empty($data['resocomplete']) ? $data['resocomplete']
    : null,
            ':resolution_closed_on'    => !empty($data['resoclosed']) ? $data['resoclosed'] : nu
    ll,
            ':cost_of_defect'         => isset($data['costofdefect']) ? (float) $data['costofde
    fect'] : null,
        ]);
    }

    /**
     * Deletes analysis entry only (db.php?function=deleteanalyze)
     */
    public function delete(int $defectId): bool
    {
        $stmt = $this->pdo->prepare('DELETE FROM `defect_analyses` WHERE `report_id` = :report_i
    d');
        $stmt->execute([':report_id' => $defectId]);
        return $stmt->rowCount() > 0;
    }
}
```